

Research Paper

Layered learning in a quadrotor drone: Simultaneous controlling and path planning using optimal fuzzy fractional order proportional integral derivative and proximal policy optimization

Hamed Shahbazi^{a,*}, Vahid Tikani^b, Roholamin Fatahi^b

^a Department of Mechanical Engineering, University of Isfahan, Isfahan, Iran

^b Department of Mechatronic Engineering, University of Isfahan, Isfahan, Iran

ARTICLE INFO

Keywords:

Deep reinforcement learning
Quadrotor dynamics
Fuzzy system
Heuristic algorithms
Control

ABSTRACT

Unmanned aerial vehicles (UAVs), particularly quadrotors, have seen a surge in applications ranging from surveillance to delivery services. However, autonomous control remains a challenge due to the complexity of their dynamics and the need for real-time responsiveness. This paper addresses the significant problem of autonomous quadrotor control by introducing a novel learning method that integrates a deep neural network with advanced control techniques. Novelty of the work is distribution of different learning tasks in different layers of learning and combining the total learning system in a applicable UAV. The purpose is to enhance the quadrotor's ability to perform complex maneuvers autonomously, which is crucial for expanding its application scope. The proposed method begins with the acquisition of direction control using a fractional order proportional–integral–derivative (FOPID) controller. The learning parameters are refined through a fuzzy method, ensuring adaptability and precision. For the optimization of the FOPID parameters, we apply two stochastic algorithms: Genetic Algorithm and Particle Swarm Optimization, comparing their efficacy. Our approach successfully teaches the quadrotor to generate and control a flight path autonomously. The second stage of learning involves proximal policy optimization, enabling the execution of intricate maneuvers. The experimental setup includes a quadrotor on a test stand with three degrees of freedom, where we validate the method through rigorous simulation and physical tests. The results demonstrate that our method not only teaches the quadrotor complex maneuvers but also ensures robust and precise execution, marking a significant advancement in UAV control systems. The implications of this research extend to improving UAV reliability for critical applications, potentially transforming the landscape of autonomous aerial operations.

1. Introduction

Due to the increasing use of unmanned aerial vehicles (UAVs) for military and civilian applications in recent decades, including fire-fighting, search, and mapping, researchers have become interested in designing and controlling these UAVs. The design of multi-rotor drones such as quadrotors has been highly regarded at the beginning of the twentieth century (Maaruf et al., 2022).

Historical and Current Landscapes of Autonomous Quadrotor Control is discussed in Asignacion and Satoshi (2024). This comprehensive review assists early-career researchers by analyzing experimental results from leading publications to identify trends and gaps in quadrotor tracking control research, highlighting the precision and speed achievements in quadrotor flights. In some research projects Reinforcement Learning and Model Predictive Controller are used (Zhang et al., 2023; Gu et al., 2024). In Li et al. (2022) the research focuses on enhancing

safe and agile navigation for quadrotors in complex environments by exploiting information collected during tasks, addressing the learning model predictive control problem.

Another deep review presented in de Oliveira Evald et al. (2024). This paper offers insights into various attitude control strategies for quadrotors, addressing the complexities and advancements in control techniques. The study in Idrissi et al. (2022) Provides an overview of quadrotor UAVs, discussing applications, design, and control algorithms, highlighting advancements and challenges in the field

Altug et al. developed quadrotor modeling based on the Euler–Newton method, focusing on sustainability and feedback using images to design and develop controllers for quadrotors (Altug et al., 2011). Dunfield and his colleagues also designed a neural network-based controller for quadrotors (Dunfield et al., 2015). Lee et al. devised a

* Corresponding author.

E-mail addresses: shahbazi@eng.ui.ac.ir (H. Shahbazi), vahid.tikani@eng.ui.ac.ir (V. Tikani), r.fattahi@eng.ui.ac.ir (R. Fatahi).

<https://doi.org/10.1016/j.engappai.2024.108926>

Received 15 February 2024; Received in revised form 6 May 2024; Accepted 1 July 2024

Available online 11 July 2024

0952-1976/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

sliding mode controller for a quadrotor (Lee et al., 2010). Bora et al. also designed a hybrid fuzzy controller to control a quadrotor (Bora et al., 2016). The design of fuzzy controllers has been widely used for dynamic systems, including the design of fuzzy controllers for quadrotors (Bora et al., 2016). The PSO optimization algorithm was used to adjust the controller parameters in the controls. Chao et al. compared genetic and PSO algorithms to optimize the control gains of the PID controller (Ou and Lin, 2006). Cheng et al. used the PSO algorithm to design trajectory tracking control in a quadrotor (Peng et al., 2013).

Another control technique for the control of a quadrotor is based on fractional order PID (FOPID). According to the literature, a wide range of methods have been reported for designing FOPID controllers. For instance, Dorcak et al. presented a state-space method based on feedback pole placement (Dorcak et al., 2002). In another research, Petras et al. introduced the FOPID design based on the pole distribution of the characteristic equation (Petras, 2000). Vinagre et al. proposed a different method based on the expected crossover frequency and phase margin (Vinagre et al., 2000).

While stochastic optimization algorithms are prevailing to achieve the universally optimal solution, particle swarm optimization (PSO) and genetic algorithm are optimization strategies from the class of natural evolutionary computation. PSO, which is used for the optimum design of PID controllers (Gaing, 2004), is a biologically inspired technique proposed by Kennedy and Eberhart (1995). A genetic algorithm has also been proposed based on the process of natural selection that simulates natural evolution such as crossover, mutation, and selection.

Deep Neural Networks (DNNs) have ushered in a transformative era in the domains of control and navigation, markedly enhancing the efficacy and functional breadth of a multitude of systems. These sophisticated networks are intricately integrated within various reinforcement learning methodologies, thereby establishing deep reinforcement learning as a formidable approach for mastering the complexities of control and navigational tasks. special types of deep neural networks such as Cohen Grossberg bidirectional associative memory neural network neural networks (Chandrasekar et al., 2022) are embedded in agents to stabilize the learning methods. It incorporates Markovian jump parameters to model the system as a continuous-time, discrete-state Markov chain. The study employs Lyapunov functional and weak infinitesimal generator to establish algebraic conditions for ISS (Radhika et al., 2023).

The cornerstone of this study is the development and validation of a complementary method for simultaneous trajectory planning and control of a quadrotor. We propose a layered learning model, detailed in Section 2, which enables the quadrotor to learn various control tasks across different layers, enhancing its adaptability and efficiency in dynamic environments. In Section 3, we delve into the dynamic model of the quadrotor, setting the stage for the introduction of two control strategies: the classical PID and the fuzzy-PID controllers. These controllers' gains are finely tuned using the Particle Swarm Optimization (PSO) algorithm, ensuring optimal performance. A comparative analysis of these controllers' effectiveness in the quadrotor's attitude control is subsequently presented, highlighting the nuanced differences and situational advantages of each approach. Furthering our exploration, we examine the efficacy of optimal-PID and Fractional Order PID (FOPID) controllers both underpinned by stochastic algorithms in the realm of attitude control. This comparative study not only sheds light on the controllers' operational capabilities but also underscores the potential of stochastic methods in achieving superior control precision. Section 3.3 is dedicated to the pivotal aspect of path planning. Here, we employ a deep reinforcement learning method to not only derive an optimal path for the quadrotor but also to impart this knowledge effectively to the UAV. This dual focus on acquisition and execution is a testament to the method's comprehensive design. The main role of deep reinforcement learning, specifically the Proximal Policy Optimization (PPO) algorithm, in the given problem is to train an agent, which

in this case is a quadrotor, to follow specified paths while being exposed to unknown disturbances. Additionally, it is used to train the quadrotor to perform acrobatic maneuvers, rewarding the quadrotor for its agility in executing these complex movements. The text emphasizes the use of a simple and intelligent design to achieve these objectives. The theoretical and practical merits of our approach are substantiated in Section 4, where we present a series of simulation results. These simulations serve as a precursor to the real-world applicability of our method, which is further corroborated in Section 5. In this final section, we introduce the experimental test bench a platform equipped with accelerometers, gyroscopes, and a microcontroller. The results obtained from this physical quadrotor not only validate our method but also demonstrate its potential to revolutionize the field of UAV control.

2. Layered learning in quadrotor

The concept of layered learning was introduced by Stone in Stone and Veloso (2000). Layered control systems are a type of control system that can increase the speed and maneuverability of systems. Control layers are designed with compatible and harmonious boards that can send and receive data with minimal bandwidth. Each board or module is a model of a relatively simple processing system (Prescott et al., 1999). The upper control layers can be effective in controlling the output of the lower layers. By launching a layered control, flexible and resistant control can be applied to dynamic systems. A control system for designing a completely automatic dynamic system should process complex and sometimes incomprehensible data from its new environment at all times (Mirtaba et al., 2023). One of the controller design patterns for this type of dynamic system is to classify and assign environmental analysis to a category of operational units. Then, it is necessary to classify the total system responses to the challenges around it into different layers (Liu et al., 2019).

Layered learning in control systems is intricately related to the concepts of distributed and decentralized decision-making, as well as fuzzy logic, which are the focus of the bibliometric study mentioned (Abbasi et al., 2022; Gonzalez et al., 2023). Layered learning, a hierarchical approach to building complex control systems, often incorporates elements of distributed decisions where control is spread across multiple nodes or layers, and decentralized decisions where each layer operates semi-independently. Fuzzy logic, frequently used within these layers, allows for handling uncertainty and imprecision, enhancing the system's ability to make nuanced decisions. The study's analysis of literature from 1995 to 2023 highlights the growing importance and interconnection of these concepts within the field (Çağlayan et al., 2024). By visualizing relationships through social network analysis, the study underscores the significant impact of these keywords on research advancements. This correlation suggests that layered learning, as an application of these principles, benefits from the collective insights and developments reported in the surveyed articles, contributing to the evolution of intelligent control systems.

The principles of this method are based on a layered structure that has been used in the construction of intelligent pilots, which has increased the level of intelligence of the quadrotor by more than a few stages. In essence, instead of being monopolized by a unique system, the controllability of the quadrotor is achieved by a system composed of several agents. All control agents are placed in a separate layer and autonomously make decisions with independent processors, memory, and peripherals. In this section, each layer has a special responsibility, and some of the responsibilities of the layers complement each other. The layered structure of this intelligent pilot has many applications and advantages. Fig. 1 shows the overall structure of this intelligent pilot and the relationships between its layers. In this figure each layer serves a unique function, from motor speed regulation to advanced navigational computations, forming an intelligent, multi-agent control structure.

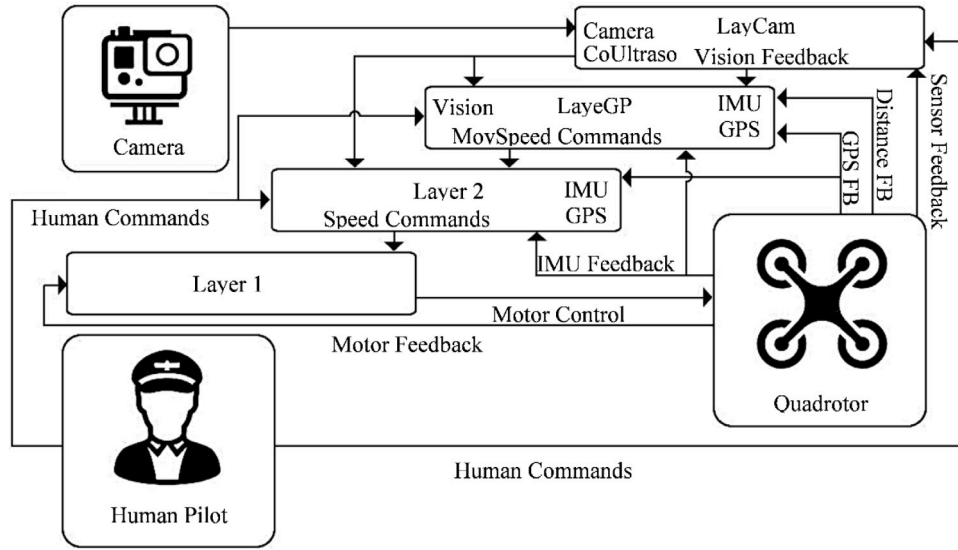


Fig. 1. Proposed layered learning structure in this paper.

The motivation behind this method is to enhance the control system's ability to process complex and dynamic environmental data efficiently. Implementing layered learning in drone technology enhances flexibility and resilience, offering several key benefits. It leads to increased speed and maneuverability as tasks are distributed across multiple layers, allowing the system to quickly adapt to environmental changes and improve the drone's agility. The approach also augments system intelligence; with each layer operating autonomously and equipped with its own processing capabilities, decision-making becomes more sophisticated. The modular nature of layered systems ensures scalability, enabling the integration of new layers or alterations to existing ones without systemic disruption. This method also ensures robustness, providing a framework capable of withstanding dynamic changes and uncertainties. Furthermore, the harmonious design of control layers guarantees compatibility, facilitating efficient data exchange and coordination, which results in a cohesive control strategy. In essence, the layered learning method is designed to foster the development of drones that are smarter, more adaptable, and efficient, particularly adept at navigating through complex and unpredictable environments.

As shown in Fig. 1 the first layer is responsible for the speed control of the motors, which can sense the electric currents and control the rotational speed. In the second layer of This structure, a group of sensors and electronic boards are used to control and manage the data and classify them. One of the most important sensors in this layer is the IMU, which is a combination of an accelerometer and a magnetometer. This sensor calculates all the angular parameters and angular velocities. This layer uses a smart processor and the desired peripherals to calculate roll, pitch, and yaw angles. It is programmed in C language code and communicates with the computer via a USB cable.

After detecting the path in the second layer, the third layer of the layered control structure calculates the roll, pitch, yaw, and angular velocity of the quadrotor for navigating the path. In processing this layer, in addition to receiving moment-by-moment information from other sensors and previous layers, the linear velocity of the quadrotor's center of mass must also be calculated and applied in the calculation of rewards and penalties in the processor layers. The processor used in this layer must be more powerful than the processors in the previous layers, and the amount of memory and other peripherals in this layer may also be greater by this amount.

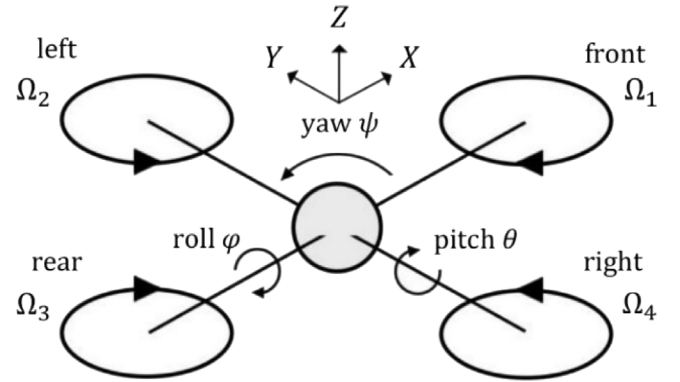


Fig. 2. Illustration of the quadrotor as a MIMO system with inter-axis coupling.

Fig. 1 shows that the fourth layer is used to learn high level tasks. In this layer, the processor calculates the motor speed at each moment after calculating the roll, pitch, yaw, and linear velocity of the quadrotor. In this layer, various learning policies such as PPO, AC, and linear and nonlinear controllers are used to process data. The processor used in this layer must be more powerful than the processors in the previous layers, and the amount of memory and other peripherals in this layer may also be greater by this amount. In this layer, the speed of each of the four quad motors is adjusted such that the quad is positioned at the desired coordinate and angle.

3. Problem modeling

3.1. Dynamic model of quadrotor

The quadrotor is defined as a Multiple Input Multiple Output (MIMO) system, demonstrating significant inter-axis coupling and underactuation. It features six degrees of freedom while being equipped with only four rotors for control. This configuration necessitates intricate control strategies to manage its movement, as illustrated in Fig. 2. The quadrotor's design integrates four rotors directly connected to its central body, enabling aerial maneuverability.

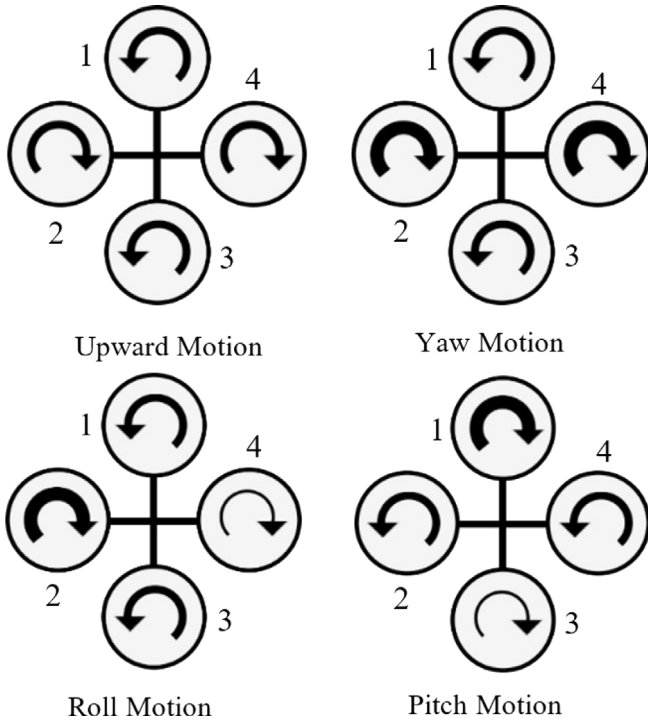


Fig. 3. Depiction of the quadrotor's propeller configuration.

As shown in Fig. 3, through changing the rotor speed, the two pairs of propellers 1, 3 and 2, 4 rotate in contrary directions. To generate vertical motion, the four propeller's speeds simultaneously increased or decrease. Roll rotation along with lateral motion is produced by changing the 2 and 4 propeller's speed. Change in 1 and 3 propeller's speed results in Pitch rotation and the corresponding lateral motion.

Yaw rotation results from the difference in the counter torque between each pair of propellers. According to literature, dynamic equations of quadrotor are producible by several methods (Sabatino, 2015). In accordance with the ideal, a model combines the nonlinear gyroscopic effects arise from the rigid body rotation in space and the propulsion rotors rotation. The nonlinear equations of motion are derived from the dynamic model is developed in Kennedy and Eberhart (1995). Gyroscopic effects arise from the propeller rotation coupled with the body rotation. The propeller gyroscopic effects for roll and pitch are respectively $J_r \dot{\phi} \Omega_r$, $J_r \dot{\theta} \Omega_r$. So the nonlinear equations are given by (Bouabdallah, 2007):

$$I_{xx} \ddot{\phi} = \dot{\theta} \psi (I_{yy} - I_{zz}) + J_r \dot{\theta} \Omega_r + l (-T_2 + T_4) + (-1)^{i+1} \sum_{i=1}^4 R_{mxi} - h \sum_{i=1}^4 H_{yi} \quad (1)$$

$$I_{yy} \ddot{\theta} = \dot{\psi} \phi (I_{zz} - I_{xx}) + J_r \dot{\phi} \Omega_r + l (-T_1 - T_3) + (-1)^{i+1} \sum_{i=1}^4 R_{myi} + h \sum_{i=1}^4 H_{xi} \quad (2)$$

$$I_{zz} \ddot{\psi} = \dot{\phi} \psi (I_{xx} - I_{yy}) + J_r \dot{\phi} \Omega_r + l (H_{x2} - H_{x4}) + l (-H_{y1} + H_{y3}) \quad (3)$$

$$m \ddot{z} = mg - (c \psi c \phi) \sum_{i=1}^4 T_i \quad (4)$$

$$m \ddot{x} = (s \psi s \phi + c \psi s \theta c \phi) \sum_{i=1}^4 T_i - \frac{1}{2} C_x A_c \rho \dot{x} |\dot{x}| \quad (5)$$

$$m \ddot{y} = (-c \psi s \phi + s \psi s \theta s \phi) \sum_{i=1}^4 T_i - \frac{1}{2} C_y A_c \rho \dot{y} |\dot{y}| \quad (6)$$

The state-space $\dot{X} = f(X, U)$ form of quadrotor model can be written from Eqs. (1)–(6) with U as inputs and X as state vector which is defined as follow (Gaing, 2004):

$$X = [\phi \ \dot{\phi} \ \theta \ \dot{\theta} \ \psi \ \dot{\psi} \ z \ \dot{z} \ x \ \dot{x} \ y \ \dot{y}] \quad (7)$$

$$U = [U_1 \ U_2 \ U_3 \ U_4] \quad (8)$$

Inputs are given by:

$$U_1 = \sum_{i=1}^4 T_i = b(\Omega_1^2 + \Omega_2^2 + \Omega_3^2 + \Omega_4^2) \quad (9)$$

$$U_2 = (-T_2 + T_4) = b(-\Omega_2^2 + \Omega_4^2) \quad (10)$$

$$U_3 = (T_1 + T_3) = b(\Omega_1^2 - \Omega_3^2) \quad (11)$$

$$U_4 = d(-\Omega_1^2 + \Omega_2^2 - \Omega_3^2 + \Omega_4^2) \quad (12)$$

So the overall quadrotor dynamic model can be described by the following equations (Bouabdallah, 2007):

$$\ddot{x} = (\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi) \frac{U_1}{m} \quad (13)$$

$$\ddot{y} = (\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi) \frac{U_1}{m} \quad (14)$$

$$\ddot{z} = -g + (\cos \phi \cos \theta) \frac{U_1}{m} \quad (15)$$

$$\ddot{\phi} = \dot{\theta} \dot{\psi} \left[\frac{I_{yy} - I_{zz}}{I_{xx}} \right] + \frac{J_r}{I_{xx}} \dot{\theta} \Omega_r + \frac{1}{I_{xx}} U_2 \quad (16)$$

$$\ddot{\theta} = \dot{\phi} \dot{\psi} \left[\frac{I_{zz} - I_{xx}}{I_{yy}} \right] - \frac{J_r}{I_{yy}} \dot{\phi} \Omega_r + \frac{1}{I_{yy}} U_3 \quad (17)$$

$$\ddot{\psi} = \dot{\theta} \dot{\phi} \left[\frac{I_{xx} - I_{yy}}{I_{zz}} \right] + \frac{1}{I_{yy}} U_4 \quad (18)$$

3.2. Learning to control attitude

Due to very good determination of dynamic model, In the attitude control layer, model based controllers are used. Learning in this layer is usually is some parameter learning of classical or nonlinear controllers. In the following, three controllers are designed for quadrotor stabilization, and their parameters will be learned using some metaheuristic methods:

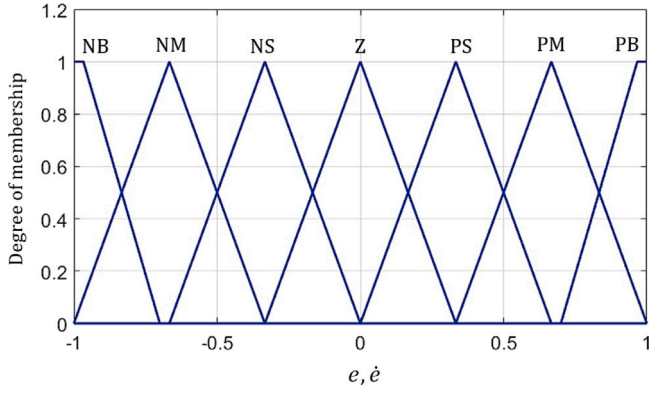
1. Classical proportional–integral–derivative (PID) controller
2. PID controller using a fuzzy system to adjust control gains
3. Fractional order PID.

As indicated, control inputs are identified by equations. Three control inputs, U_2 , U_3 and U_4 are produced to control the quadrotor's attitude, and they control Roll, Pitch, and Yaw motions.

In the following, two controllers are designed for quadrotor stabilization; classical proportional–integral–derivative (PID) controller and PID controller using a fuzzy system to adjust control gains. Three control inputs, U_2 , U_3 and U_4 are produced to control the quadrotor's attitude, and they control Roll, Pitch, and Yaw motions.

Classical-proportional–integral–derivative controller is a type of feedback-based controller that is used predominantly in control systems. Due to the simple structure and robust performance of this type of controller, its design and implementation is very common in the industries. The general relationship governing on the classical-proportional–integral–derivative controller is as follows:

$$u(t) = K_p e(t) + K_I \int_0^t e(t) dt + K_D \frac{d}{dt} e(t) \quad (19)$$

Fig. 4. Membership function for e and $\dot{e}$.

The control gain, K_p , is the control feedback with respect to the error between the optimal amount and the actual value. Derivative control gain, K_D , is the control feedback to the error rate, and integral control gain, K_I , is control feedback to the sum of the error values which included in the control input. For attitude control of the quadrotor, three U_2 , U_3 and U_4 control inputs are produced to control the direction of Roll, Pitch, and Yaw.

3.2.1. Fuzzy gain learning

The design of a PID controller with the desired performance, depends on the leaning of the controller gain values. In the next step, a fuzzy system has been designed to determine the control gain instantaneously. A fuzzy controller operates based on the PID controller and modifies the control gain values based on a fuzzy logic instantaneously, so that the system has a favorable response. Using predetermined fuzzy rules, and by taking error parameters and error derivative at any instant as inputs, the PID fuzzy system changes the values of controller's gain.

Fuzzy controller rules define the relationship between the error and error derivative with the control gain coefficient. As shown in Fig. 4, seven parameters are assigned to define the error membership functions and error derivative in the fuzzy system, and with this, the numerical values are converted to fuzzy definitions.

In order to estimate the control gains, three values of α_p , α_D , and α_I are considered as the fuzzy system output within the range of $[-1,1]$. Then, using Eqs. (20)–(22), the control gains are obtained between the considered upper and lower limits.

$$K_p = (K_{p,max} - K_{p,min}) \alpha_p + K_{p,min} \quad (20)$$

$$K_D = (K_{D,max} - K_{D,min}) \alpha_D + K_{D,min} \quad (21)$$

$$K_I = (K_{I,max} - K_{I,min}) \alpha_I + K_{I,min} \quad (22)$$

As shown in Fig. 5, the parameters S and B are used to define the fuzzy system output membership functions.

The most important part of designing fuzzy controller-PID is the definition of linguistic rules. Linguistic rules specify the conditions for each output with respect to the quality of input values, e and $\dot{e}$. α_p , α_I and α_D values are outputs of the fuzzy system obtained with the rules set forth in Tables 1 and 2. The first table describe fuzzy rules for learning the α_p and α_I values, These rules specify the conditions for each output concerning the quality of input values, e and $\dot{e}$. Table 2 describe fuzzy rules for learning the α_D value, The values of α_p , α_I , and α_D are obtained from the fuzzy system using the rules.

In Fig. 6-a, α_p and α_I which are the outputs of the fuzzy system, are plotted as surface representation based on the input values, e and $\dot{e}$. A surface representation of output α_D in terms of inputs in the fuzzy system is also plotted in Fig. 6-b.

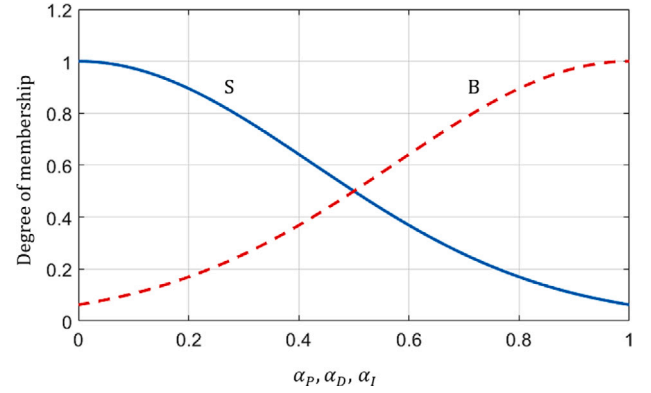
Fig. 5. Membership function for α_p , α_I and α_D .

Table 1

Fuzzy rules for learning the α_p and α_I values.

$\dot{e}$								
PB	PM	PS	ZO	NS	NM	NB		
B	B	B	B	B	B	B	NB	e
S	B	B	B	B	B	S	NM	
S	S	B	B	B	S	S	NS	
S	S	S	B	S	S	S	ZO	
S	S	B	B	B	S	S	PS	
S	B	B	B	B	B	S	PM	
B	B	B	B	B	B	B	PM	

Table 2

Fuzzy rules for learning the α_D value, The values of α_p , α_I , and α_D are obtained from the fuzzy system using the rules.

$\dot{e}$								
PB	PM	PS	ZO	NS	NM	NB		
S	S	S	S	S	S	S	NB	e
B	S	S	S	S	S	B	NM	
B	B	S	S	S	B	B	NS	
B	B	B	S	B	B	B	ZO	
B	B	S	S	S	B	B	PS	
B	S	S	S	S	S	B	PM	
S	S	S	S	S	S	S	PM	

3.2.2. Fractional calculus

Fractional calculus is a branch of mathematical analysis that studies the possibility of taking real number power of the differential operator and integration operator (Tarasov, 2019). There are several definitions of fractional derivatives. The definition for Caputo's fractional derivative of order λ of a function $f(t)$ is given by:

$${}_a D_t^\alpha f(t) = \frac{1}{\Gamma(\alpha - n)} \int_a^t \frac{f^{(n)}(\tau)}{(t - \tau)^{\alpha - n + 1}} d\tau, \quad n - 1 < \alpha < n \quad (23)$$

The Grünwald–Letnikov and the Riemann–Liouville definitions of a fractional derivative are as follows:

$${}_a D_t^\alpha f(t) = \lim_{h \rightarrow 0} \frac{1}{h^\alpha} \sum_{k=0}^{\left[\frac{t-a}{h} \right]} (-1)^k \binom{\alpha}{k} f(t - kh) \quad (24)$$

$${}_a D_t^\alpha f(t) = \frac{1}{\Gamma(\alpha - n)} \frac{d^n}{dt^n} \int_a^t \frac{f(\tau)}{(t - \tau)^{\alpha - n + 1}} d\tau, \quad n - 1 < \alpha < n \quad (25)$$

where h refers to step time increment and Γ is Euler's gamma function. a and t are the limits related to the operation of fractional differentiation.

3.2.3. Discretization methods

There are two discretization methods consist of direct discretization and indirect discretization. Several direct discretization methods by

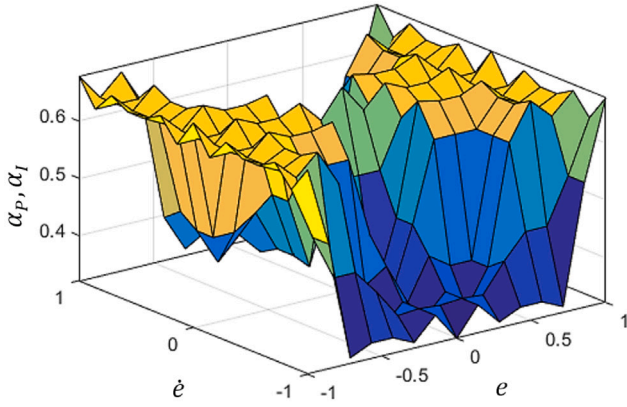
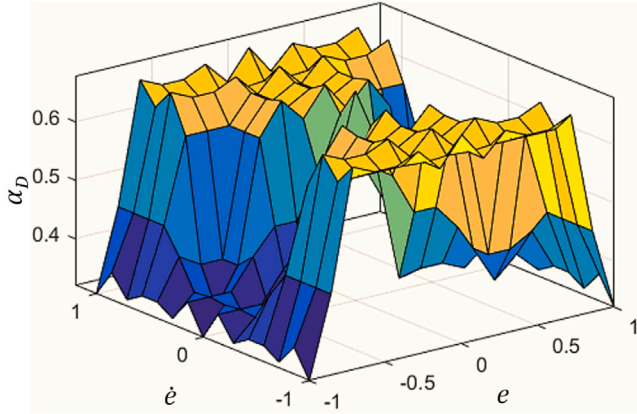
(a) Fuzzy inference system output surface for α_P, α_I (b) Membership function for α_D

Fig. 6. Fuzzy inference system.

finite element and difference equation are proposed like short memory principle and Lagrange function interpolation method.

With respect to Grunwald–Letnikov definition, by generating function $s = \omega(z^{-1})$, it is possible to transform the fractional order differentiator s^α from s domain to z space by Euler and Tustin method. For achieving the coefficient of approximation equation for fractional calculus, Tustin operator as the generating function is considered as follows:

$$s^\alpha = (\omega(z^{-1}))^\alpha = \left(\frac{2}{T} \cdot \frac{1 - z^{-1}}{1 + z^{-1}} \right)^\alpha \quad (26)$$

And perform the power series expansion of Eq. (26) The discretized result is:

$$Z \{D^\alpha x(t)\} = CFE \left\{ \left(\frac{2}{T} \cdot \frac{1 - z^{-1}}{1 + z^{-1}} \right)^\alpha \right\} X(z) \approx \left(\frac{2}{T} \right)^\alpha \frac{P_p(z^{-1})}{Q_q(z^{-1})} X(z) \quad (27)$$

where $CFE(x)$ is the continued fraction expansion of x ; p and q are the orders of the approximations; P and Q are polynomials of degrees p and q .

The fractional order controller can be approximated by discretization methods as follows:

$$G_c(z) = k_p + k_i w_i(z) + k_d w_d(z) \quad (28)$$

where $w_i(z)$ and $w_d(z)$ are respectively the discrete approximation equation of fractional order integral $s^{-\lambda}$ and the discrete approximation equation of s^δ .

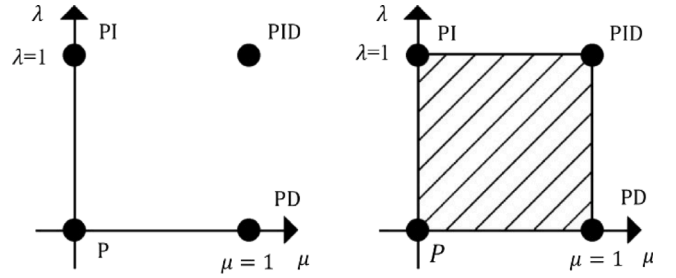


Fig. 7. PID controllers with fractional orders.

3.2.4. Fractional order controllers

In Fact, a PID controller is a type of feedback controller that is widely used in control systems. PID controllers control systems by generating control signals with respect to the error between measured data and desired set point. The transfer function of the PID controller is given by:

$$G_c(s) = K_p + K_i \frac{1}{s} + K_d s \quad (29)$$

The proportional gain (K_p) determines the reaction to the error between measured data and desired set point, differential gain (K_d) determines the reaction to the rate of changing the error and integral gain (K_i) determines the reaction to the summation of errors.

To improve the performance of the PID controllers using a fractional order controller are suggested which are involved by an integer of order λ and a differentiator of order μ . The differential equation of fractional order $PI^\lambda D^\mu$ is described as:

$$u(t) = K_p e(t) + K_i D^{-\lambda} e(t) + K_d D^\mu e(t) \quad (30)$$

The transfer function of FOPID obtained through Laplace transform is given by:

$$G_c(s) = K_p + T_i s^{-\lambda} + T_d s^\mu \quad (\lambda, \mu > 0) \quad (31)$$

As shown in Fig. 7, $\lambda = 1$ and $\mu = 1$ indicates normal PID controller; for $\lambda = 0$ and $\mu = 1$ FOPID controller changes to the normal PD controller; $\lambda = 1$ and $\mu = 0$ indicates normal PI controller and $\lambda = 0$ and $\mu = 0$ indicates a proportional gain.

From Eq. (30) to design a fractional order PID controller for roll, pitch and yaw control the five parameters K_p, K_i, K_d and λ, δ are required to be designed for each attitude controller. Because the dynamic of quadrotor is highly coupled, all the parameters must be designed simultaneously according to control objectives. For the conventional PID controller design, all the poles of the closed-loop transfer function must be confined to the left half of the s plane. Based on the stability condition of the fractional order system, a controller, which stabilizes the integer order system could stabilize the fractional order system.

3.2.5. Optimization algorithms

Learning the parameters of the Fuzzy and fractional order PID controller has been performed by two well-known stochastic optimization algorithms, namely the GA and PSO algorithms. The process of optimizing the controller parameters is shown in Fig. 8. In the following subsections, a brief description of these two learning algorithms is presented.

Genetic algorithm: Genetic algorithm is one of the optimization stochastic algorithms which is inspired by natural evolution (Sivanandam et al., 2008). The algorithm is used to minimize or maximize an objective function. The genetic algorithm is initialized with the creation of a random population of solution vectors. The genetic algorithm has three main operators consist of reproduction, crossover, and mutation;

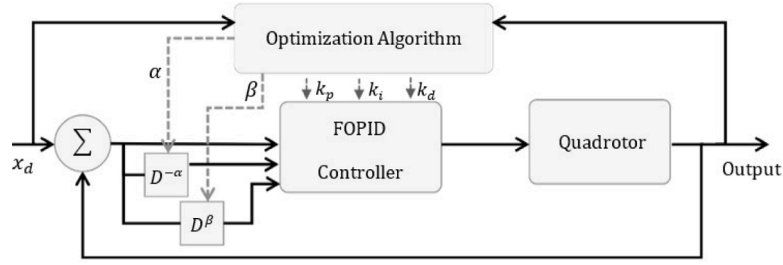


Fig. 8. Block diagram of the optimization algorithm-based FOPID controller for quadrotor.

to achieve the best population of the solution, in each iteration, every solution vectors are subjected to these three operators that results in a better population of solution vectors in the next iteration.

Particle Swarm Optimization: Particle swarm optimization is a stochastic optimization algorithm, inspired by social behavior of fish or bird schooling (Wang et al., 2018). Different from GA algorithm there is not any operators such as crossover, reproduction, and mutation in the PSO algorithm. The PSO algorithm is initialized with the creation of particles distributed randomly over the whole solution domain. The particles move in each iteration with respect to the gbest (the best value found so far) and the pbest (individual particle's best position). The fitness of particles for a particular position is evaluated by the objective function.

By finding the two best values the velocity in each dimension in the sequential iteration for each particle is updated by the following equations:

$$v_i(t+1) = v_i(t) + c_1\phi_1(p_i(t) - x_i(t)) + c_2\phi_2(p_g(t) - x_i(t)) \quad (32)$$

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (33)$$

where v_i is the particle velocity, x_i is the particle position. P_i is individual particle's best position and P_g is the best value found so far. c_1 and c_2 are two constant which implies cognitive learning and the social learning rate respectively. The values of the two constants stand for the comparative importance of the learning of the particles from its own best position (p_i) and the global best position (p_g).

Due to the lacks of velocity control mechanism in the original PSO formula, it does not have a proper ability to search at a fine grain (Wang et al., 2018).

$$v_{id}(t+1) = \omega v_{id}(t) + c_1\phi_1(p_i(t) - x_i(t)) + c_2\phi_2(p_g(t) - x_i(t)) \quad (34)$$

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (35)$$

3.2.6. Parameters learning using Taguchi method

Parameter learning has a significant effect on the performance of meta-heuristic algorithms. In this research, Taguchi method is applied to determine the proper parameters. This method is developed by Taguchi. In details, it categorizes the factors into two groups including signal and noise factors and utilizes a fractional factorial experiment instead of full factorial experiments. The method aims at minimizing the effect of noise to specify the optimal level of the signal factors. To this end, the signal to noise ratio (S/N) is applied that estimates the amount of variation of the response.

This fuzzy method can enhance the FOPID controller by allowing it to handle uncertainties and nonlinearities effectively. The fuzzy system dynamically adjusts the control gain values based on error parameters and their derivatives, leading to a more responsive and adaptive control strategy.

In this study, the L9 design of the Taguchi is utilized and the signal to noise ratio (S/N) is considered which estimates the amount

Table 3
Levels of parameters of PSO algorithm.

Parameter	Level 1	Level 2	Level 3
Crossover percentage	0.60	0.70	0.80
Mutation percentage	0.10	0.15	0.20
Population size	50	70	90

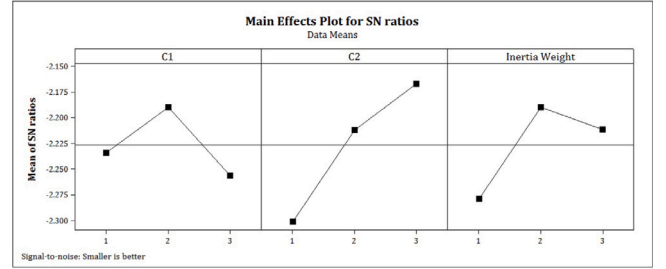


Fig. 9. The proper values of the parameters determined by Minitab.

of variation of the response. Due to minimization nature of the cost function of the controller response, “Smaller is better” type is desirable. Moreover, to change the cost function values to non-scale data, the relative percentage deviation (RPD) formulation is used as follows:

$$RPD = \frac{|Alg_{sol} - Best_{sol}|}{|Best_{sol}|} \times 100 \quad (36)$$

where, Alg_{sol} is the objective function value obtained by the meta-heuristic algorithms and $Best_{sol}$ is the optimal solution obtained for each instance by the GAMS software.

The ability to explore is determined by the societal factor and individual factor. The learning factor C_1 and C_2 and weight W are three adjustable parameters in PSO. Based on the three parameters, the considered levels of the parameters are listed in Table 3. The table presents three parameters of the Particle Swarm Optimization (PSO) algorithm across three levels. The crossover percentage increases from 60% at level 1 to 80% at level 3, indicating a higher chance of combining solutions from the population. The mutation percentage shows a growth from 10% to 20%, suggesting an increase in the diversity of the population by altering solutions. Lastly, the population size expands from 50 to 90, which could potentially enhance the search for optimal solutions by using a larger group of candidate solutions. These parameters are crucial for the algorithm's performance.

Using Minitab Software 16.2, the proper values of the parameters are determined. The highest value of the S/N ratio in Fig. 9 indicates the higher performance level for each parameter.

3.2.7. Objective function

An appropriate objective function should be defined for designing the Fuzzy and FOPID controller. The objective function could be defined based on desired specifications, for instance, overshoot (M_p),

rise time (t_r), steady-state error (e_{ss}) and settling time (t_s) are some significant items in the time domain which show the performance of a controller.

There are several criterions of control quality to evaluate the performance of controller, including integral of squared-error (ISE), integrated absolute error (IAE) and the integrated of time-weighted-squared-error (ITSE) that are considered in the step input for control design. The ISE, IAE and ITSE performance criteria formulas are given by:

$$ISE = \int_0^{\infty} e^2(t) dt \quad (37)$$

$$IAE = \int_0^{\infty} |e(t)| dt \quad (38)$$

$$ITSE = \int_0^{\infty} te^2(t) dt \quad (39)$$

These three criteria formulas have some advantages and disadvantages, for instance, the minimization using IAE and ISE criteria may result in a response with relatively small overshoot but a long settling time. On the other hand derivation processes of the ITSE formula are complex and time-consuming. In the paper, another time domain performance criterion is defined by:

$$J_i = e^{-\gamma}(M_p + e_{ssi}) + (1 - e^{-\gamma})(t_{si} + t_{ri}) \quad (40)$$

$$J(k) = \sum J_i \quad (41)$$

where K is the parameter vector $[K_{p\theta}, K_{i\theta}, K_{d\theta}, \mu_{\theta}, \lambda_{\theta}, K_{p\varphi}, K_{i\varphi}, \dots]$, and γ is the weighting factor. It is up to the user to set the γ properly to achieve the desired specification. In this study, the value of β is set to 1.4 so the effect of each parameter is the same in the objective function. It is worth mentioning that since the dynamic of quadrotor is highly coupled so all the parameters are designed simultaneously and the objective function is equal to the summation of the J_i for the roll, pitch and yaw response to the step input.

3.2.8. Learning in FOPID

A multitude of approaches have been explored for parameter learning in PID and FOPID controllers. The primary objective of this research is to engineer optimal PID and FOPID controllers utilizing stochastic algorithms. To this end, the Particle Swarm Optimization (PSO) and Genetic Algorithm (GA) are implemented to determine the gain values and parameters for both controllers. A ceiling of 300 iterations is set for each algorithm, and the parameters for each are derived using the Taguchi method. The efficacy of PSO and GA optimized PID and FOPID controllers in managing the quadrotor's attitude has been rigorously evaluated and juxtaposed.

The quadrotor's control system receives a step input to regulate roll, pitch, and yaw. The corresponding responses are illustrated in Figs. 10, 11, and 12 for roll, yaw, and pitch control, respectively. A summary of the comparative outcomes for roll control is encapsulated in Table 4. The PSO-FOPID and GA-FOPID controllers demonstrate superior performance over their PID counterparts in both roll and pitch control, with PSO-FOPID achieving enhanced control by further minimizing the cost function. Conversely, in yaw control, the GA-FOPID controller exhibits a more favorable performance than the PSO-FOPID controller.

The Table 4 presents a comparison of optimized PID and FOPID controllers for roll control of the quadrotor. The GA PID controller parameters are $K_p = 0.96$, $K_D = 0.14$, $K_I = 0.56$, resulting in a maximum overshoot (M_p) of 18.12%, a rise time (t_r) of 0.26 s, and a settling time (t_s) of 0.77 s, with no steady-state error (E_{ss}). The GA FOPID controller, with $K_p = 0.59$, $K_D = 0.91$, $K_I = 0.35$, $\lambda = 0.56$, $\mu = 0.63$, achieves a lower M_p of 15.1%, a t_r of 0.29 s, and a t_s of 0.896 s. The PSO PID controller has parameters $K_p = 0.95$, $K_D = 0.14$, $K_I = 0.58$, with a M_p of 19.86%, a t_r of 0.26 s, and a t_s of 0.806 s. The

Table 4

Comparative results of optimized PID and FOPID controllers for roll control of the quadrotor.

Controller	K_p	K_D	K_I	λ	μ	M_p (%)	t_r	t_s	E_{ss}
GA PID	0.96	0.14	0.56	1	1	18.12	0.26	0.77	0
GA FOPID	0.59	0.91	0.35	0.56	0.63	15.1	0.29	0.896	0
PSO PID	0.95	0.14	0.58	1	1	19.86	0.26	0.806	0
PSO FPOID	0.51	0.76	0.04	0.52	0.68	9.76	0.3	0.756	0

Table 5

Comparative results of optimized PID and FOPID controllers for yaw control of the plant.

Controller	K_p	K_D	K_I	λ	μ	M_p (%)	t_r	t_s	E_{ss}
GA PID	0.85	0.19	0.80	1	1	22.38	0.36	1.093	0.0003
GA FOPID	0.58	0.92	0.48	0.41	0.91	22.76	0.31	0.95	0.0004
PSO PID	0.83	0.193	0.80	1	1	24.83	0.34	1.043	0.0002
PSO FPOID	0.28	0.74	0.11	0.35	0.78	12.77	0.35	0.92	0.001

Table 6

Comparative results of optimized PID and FOPID controllers for pitch control of the plant.

Controller	K_p	K_D	K_I	λ	μ	M_p (%)	t_r	t_s	E_{ss}
GA PID	0.81	0.12	0.71	1	1	13.95	0.31	1.37	0
GA FOPID	0.92	0.90	0.09	0.60	0.63	6.31	0.3	0.698	0
PSO PID	0.80	0.12	0.72	1	1	14.65	0.31	1.3070	0
PSO FPOID	0.54	0.67	0.29	0.40	0.72	9.08	0.33	1.34	0

PSO FPOID controller, with $K_p = 0.51$, $K_D = 0.76$, $K_I = 0.04$, $\lambda = 0.52$, $\mu = 0.68$, shows a significantly lower M_p of 9.76%, a t_r of 0.3 s, and a t_s of 0.756 s.

Similarly, the Table 5 compares the same controllers for yaw control. The GA PID controller parameters are $K_p = 0.85$, $K_D = 0.19$, $K_I = 0.80$, with a M_p of 22.38%, a t_r of 0.36 s, and a t_s of 1.093 s, with an E_{ss} of 0.0003. The GA FOPID controller, with $K_p = 0.58$, $K_D = 0.92$, $K_I = 0.48$, $\lambda = 0.41$, $\mu = 0.91$, has a slightly higher M_p of 22.76%, a reduced t_r of 0.31 s, and a t_s of 0.95 s, with an E_{ss} of 0.0004. The PSO PID controller has parameters $K_p = 0.83$, $K_D = 0.193$, $K_I = 0.80$, resulting in a M_p of 24.83%, a t_r of 0.34 s, and a t_s of 1.043 s, with an E_{ss} of 0.0002. The PSO FPOID controller, with $K_p = 0.28$, $K_D = 0.74$, $K_I = 0.11$, $\lambda = 0.35$, $\mu = 0.78$, achieves a lower M_p of 12.77%, a t_r of 0.35 s, and a t_s of 0.92 s, with an E_{ss} of 0.001.

In both roll and yaw control, the FOPID controllers demonstrate improved performance over the PID controllers, particularly in terms of reducing overshoot and settling time.

The Table 6 presents a comparative analysis of optimized PID and FOPID controllers for pitch control. The GA PID controller, with parameters $K_p = 0.81$, $K_D = 0.12$, and $K_I = 0.71$, achieves a maximum overshoot (M_p) of 13.95%, a rise time (t_r) of 0.31 s, a settling time (t_s) of 1.37 s, and a steady-state error (E_{ss}) of 0. In comparison, the GA FOPID controller shows improved performance with $K_p = 0.92$, $K_D = 0.90$, $K_I = 0.09$, $\lambda = 0.60$, $\mu = 0.63$, a lower M_p of 6.31%, a comparable t_r of 0.3 s, and a reduced t_s of 0.698 s. Similarly, the PSO PID controller parameters are $K_p = 0.80$, $K_D = 0.12$, $K_I = 0.72$, resulting in a M_p of 14.65%, a t_r of 0.31 s, and a t_s of 1.3070 s. The PSO FPOID controller, with $K_p = 0.54$, $K_D = 0.67$, $K_I = 0.29$, $\lambda = 0.40$, $\mu = 0.72$, shows a M_p of 9.08%, a t_r of 0.33 s, and a t_s of 1.34 s. The results indicate that the FOPID controllers, particularly the GA FOPID, have a significant advantage in reducing overshoot and settling time, demonstrating their effectiveness in pitch control.

3.3. Learning to plane a path

3.3.1. Proximal policy optimization

Now, one of the most advanced algorithms in deep reinforcement learning is Proximal Policy Optimization (PPO). This developed algorithm is capable of maintaining a proper balance between performance

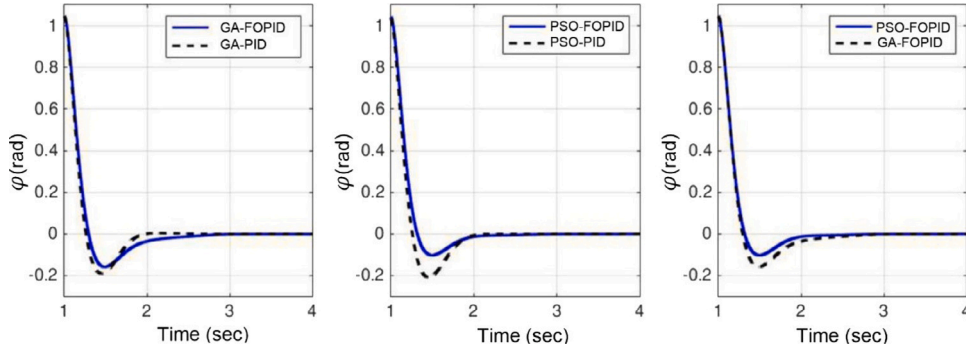


Fig. 10. Step response of roll control.

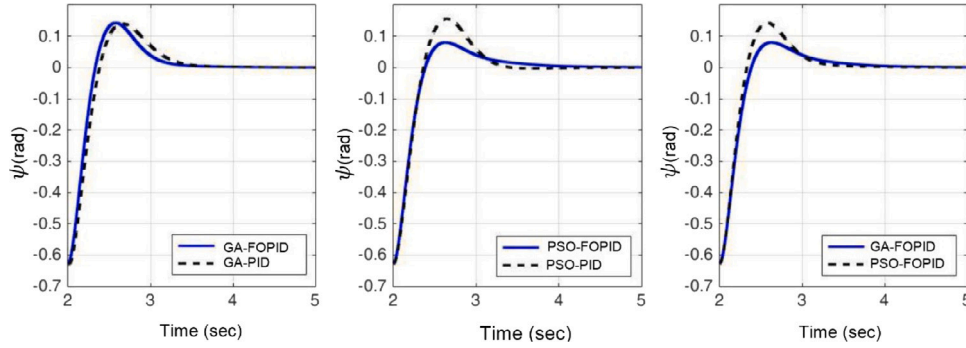


Fig. 11. Step response of yaw control.

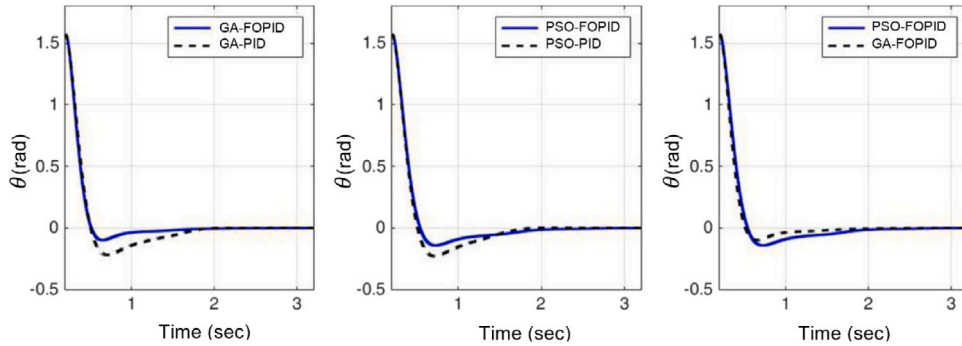


Fig. 12. Step response of pitch control.

and understanding (Schulman et al., 2017). However, it is so simple to use by a wide range of users that it cannot be used for every reinforcement learning algorithm. Because this algorithm is used to train policy-based models.

This algorithm tries to put policy changes in a range that, in addition to improvement, is not too far from the previous policy. With this approach, this algorithm succeeds in improving the policy while maintaining training stability. One of the most famous methods that also exists in this algorithm is the Actor-Critic method. In this method, an actor model is responsible for making decisions about the next action based on the current state, and the critic model evaluates the performance value of the actions taken. This algorithm allows the actor's action to improve and at the same time use criticism for evaluation, which will be discussed further.

In this project, PPO is used to train an agent that can follow specified paths while exposed to unknown disturbances. Also, the quadrotor

has been trained to perform acrobatic maneuvers. With a relatively simple and intelligent design, it is acceptable to reward a quadrotor with agile maneuvers. The structure of the method is shown in Fig. 13.

To produce a path for quadrotor, step-by-step actions must be taken, which are as follows:

In this section, an attempt is made to move the quadrotor on a reference path, and this path can be designed in such a way that the quadrotor performs maneuvers. However, the conditions for performing the maneuver depend on the power, speed, and acceleration of the quadrotor motors. Tracking the reference path while in the radius and reach of unknown disturbances is one of the tasks of aerial maneuvers, and it is assumed that the quadrotor has access to this path. The reference path has properties and features, one of the most important of which is to guarantee that this path and its dynamic maneuvers are possible in reality no matter how difficult and challenging they may

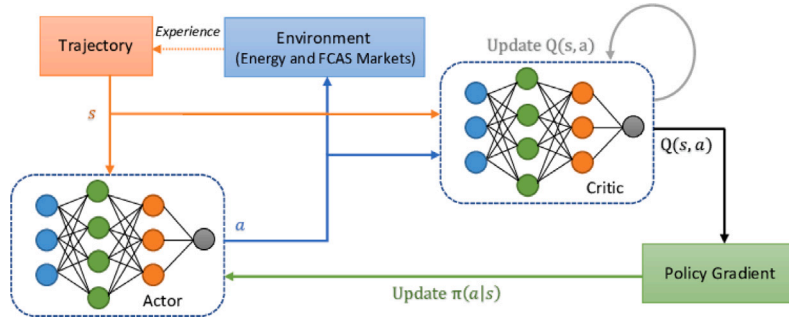


Fig. 13. Illustration of a quadrotor agent trained using PPO to follow specified paths and perform acrobatic maneuvers.

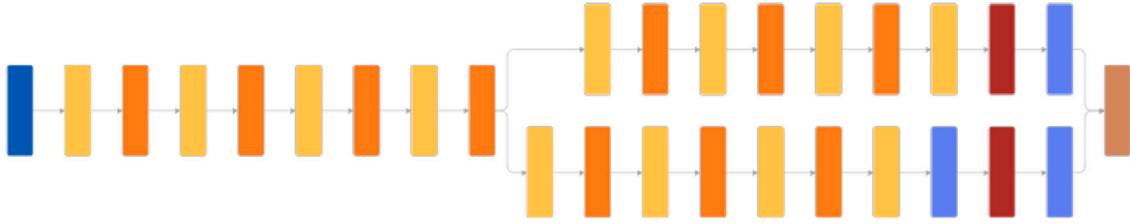


Fig. 14. Illustration of a neural network architecture for policy (actor) and value (critic) functions.

be. To perform aerial maneuvers, we must ensure that the reference path is designed in such a way that, for example, in a circular path, the tangential speed must be faster than the predetermined boundary so as to prevent free fall at the top of the circular maneuver.

$$\|v_t\| > \epsilon \sqrt{rg} \quad (42)$$

where r is the symbol for the radius of the circular path and g is the amount of gravity.

States and Actions: State s defines the position, velocity, and acceleration errors of the quadrotor with respect to the reference path, as well as the orientation and angular velocity vectors. In this error-based state, instead of informing the quadrotor with a state that has current position coordinates and target position coordinates, the desired actions are made possible with less regard to the flow paths and specific targets. The state is defined as follows:

$$s = [e_{pos}, e_{vel}, e_{acc}, q, \omega]^T \quad (43)$$

where q is a four-parameter vector that represents the quadrotor's orientation, and ω represents the angular velocity.

Actions are forces applied to the quadrotor by electric motors. To converge faster to the path and perform real-world scenarios, actions are limited between 0 and 15 N.

Network Creation: The policy (actor) and value (critic) functions are created by a neural network that maps a state s and a goal g to a distribution over actions π . In the actor network, which consists of 29 layers, inputs are processed by 4 hidden layers, each with 192, 256, 256, and 192 neurons, respectively. ReLU activation is used for all hidden units. Then, in two branches of the neural network, each of which is processed by 3 other hidden layers called μ and σ , the action distribution is processed as shown in Fig. 14. This figure shows the actor network. The critic network is shown in Fig. 15.

Then, the action is sampled through Gaussian assumption. For the main path, the $\tanh$ dual activation function is used to soften the initial response of the network. This makes it possible to train faster by giving a higher probability of selecting an action close to the desired movement, which is its initial requirement for quadrotor stability. After

the quadrotor is stable, learning more complex tasks such as path tracking and disturbance rejection becomes much easier. For the critic network, which has 10 layers, 4 hidden layers similar to the common path used in the actor network are used. Then, the output is activated by a complex function for the final variance output.

Rewards: To find the path, there are four sentences that encourage the quadrotor to follow the reference path while under the influence of random force magnitudes and directions. The reward shapes the behavior of the quadrotor to reduce position error, velocity error, acceleration error, and deviation error. Position, velocity, and acceleration rewards, r_{pos} , r_{vel} , r_{acc} , encourage the agent to synchronize itself with the position, velocity, and acceleration of the reference path.

The reward shaping for the quadrotor is defined as follows:

$$r_{pos} = -e^{-\left(\frac{1}{0.5 \text{ m}} \|e_{pos}\|\right)^2} \quad (44)$$

$$r_{vel} = -e^{-\left(\frac{1}{1 \text{ m/s}} \|e_{vel}\|\right)^2} \quad (45)$$

$$r_{acc} = -e^{-\left(\frac{1}{1 \text{ m/s}^2} \|e_{acc}\|\right)^2} \quad (46)$$

To maintain yaw stability, the yaw reward r_{ψ} is added to encourage the quadrotor to keep ψ always zero.

$$r_{\psi} = -e^{-\left(\frac{1}{5} \|\psi\|\right)^2} \quad (47)$$

Finally, the total reward is written as a weighted sum of the above rewards:

$$r = 0.6r_{pos} + 0.1r_{vel} + 0.1r_{acc} + 0.2r_{\psi} \quad (48)$$

As shown in Fig. 16, reference paths are randomly designed while taking into account the physical constraints of the quadrotor.

Early Termination (ET): To find a path and perform aerial maneuvers, tasks can be modeled as an episodic operation (segmented). However, during training, the episode ends after the agent reaches the goal or certain conditions occur. For example, if the agent deviates too much (e.g., 1 m) from the reference path, the episode ends and the episode

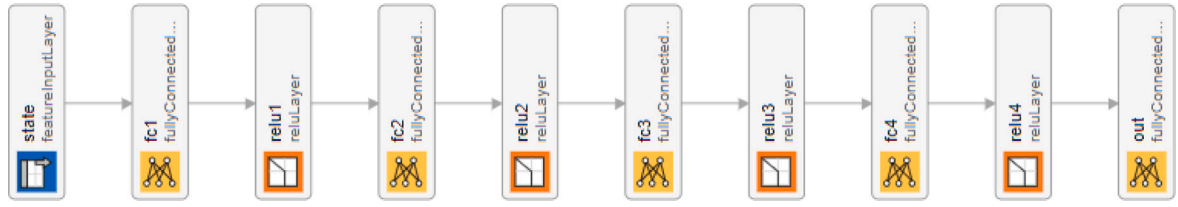


Fig. 15. Illustration of a critic network with 10 layers, including 4 hidden layers following a similar architecture as the actor network.

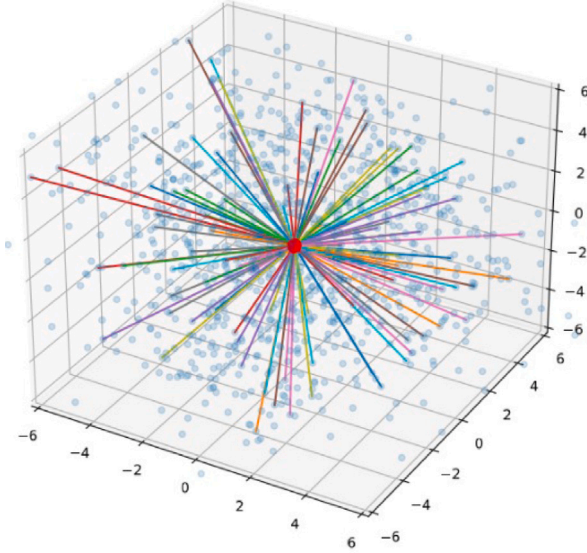


Fig. 16. Illustration of randomly designed reference paths for a quadrotor, considering its physical constraints.

is restarted. To find a path, when ψ is greater than a certain value, the end conditions are created differently.

For aerial maneuvers, the z -axis of the body, represented by z_b , is tracked during training, and when the angle between z_b and the desired acceleration vector deviates from a small value, the episode ends. These early termination conditions ensure that the agent is only exposed to relevant tasks and ultimately learns in an appropriate training space. Also, if necessary, the x -axis of the body, represented by x_b , is tracked and maintained along the reference path.

Aerial Maneuvers: To perform aerial maneuvers, it is important to expose the agent to the relevant reaction and behavior to learn the desired outcome. To do this, the reward design must be modified. That is, in addition to the main reward, secondary rewards r_{xb} and r_{zb} should be used.

$$r_{xb} = -e^{-\frac{1}{30} \|e_{xb}\|^2} \quad (49)$$

$$r_{zb} = -e^{-\frac{1}{30} \|e_{zb}\|^2} \quad (50)$$

where e_{xb} and e_{zb} represent the angle between the desired speed and the desired acceleration vector along the reference path, respectively. This ensures that during training, when z_b follows the direction of the acceleration vector, it causes the agent to receive more reward, which is eager to orient towards circular paths for aerial maneuvers. Also, matching the directions of x_b and the velocity vector ensures proper heading of the quadrotor.

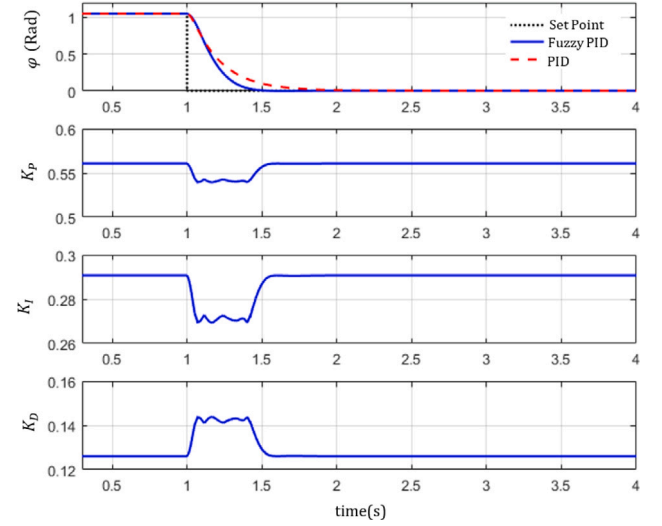


Fig. 17. Roll control and gain values changes per time.

Table 7

Comparison of attitude control using PID and Fuzzy PID controller.

t_s	PO (%)	e_{ss}	t_r	Angle	Controller
0.64	0	0	0.65	φ	PID
0.54	0	0	0.55	θ	
0.48	15.21	0	0.23	ψ	
0.38	0	0	0.39	φ	Fuzzy-PID
0.37	0.83	0	0.40	θ	
0.41	0.88	0	0.42	ψ	

4. Simulation results

4.1. Attitude control simulation

Using a fuzzy system to adjust the control gains, PID controller was simulated in MATLAB software, in order to compare and assess the performance of this controller with the classical PID controller. Fig. 17 shows the simulation result for attitude control of Roll direction, the response of the fuzzy-PID controller and PID to the step input and control gains' changes over time are drawn in this figure. It can be seen that the system response with fuzzy-PID control method is more favorable than PID.

Fig. 18 shows the simulation result for the angular control of Pitch and changes in control gains by the fuzzy controller.

Fig. 19 shows the simulation result of Yaw angle control in response to the step input for PID and fuzzy PID controller, and as well as the changes in control gains.

Table 7 compares the attitude control for PID and fuzzy PID controllers in response to the step input. Step-response characteristics includes settling time (t_s), the maximum overshoot (PO), energy storage systems (E_{ss}), and rise time (t_r) are presented in this table.

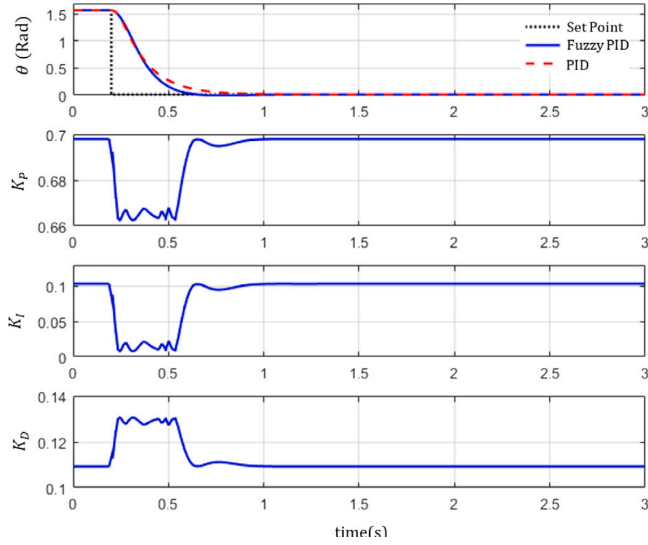


Fig. 18. Simulation outcomes for pitch angle management.

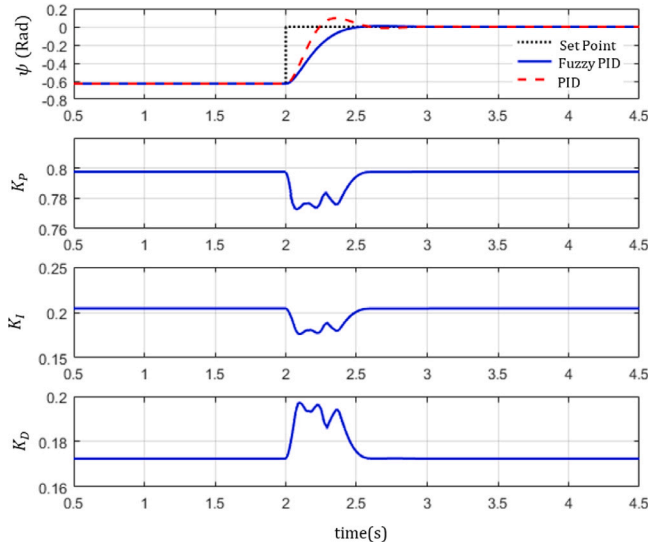


Fig. 19. Psi control and gain values changes per time.

The Table 7 compares the performance of PID and Fuzzy-PID controllers in attitude control by evaluating key step-response metrics. The PID controller shows no overshoot (%PO) and steady-state error (e_{ss}) for all angles, with settling times (t_s) of 0.64, 0.54, and 0.48 s for angles ϕ , θ , and ψ respectively. The rise times (t_r) closely match the settling times. In contrast, the Fuzzy-PID controller exhibits a minimal overshoot for angles θ and ψ , with values of 0.83% and 0.88% respectively, while maintaining a zero steady-state error. The settling times are slightly reduced to 0.38, 0.37, and 0.41 s for the respective angles, indicating a faster response. The rise times are again similar to the settling times. Overall, the Fuzzy-PID controller demonstrates a marginally faster response with negligible overshoot, suggesting a more refined control compared to the traditional PID controller, especially for the angles θ and ψ .

4.2. PPO simulations

Then, for reinforcement learning, the agent learns to track the path while exposed to unknown disturbances. Disturbance is modeled as a force that is randomly directed and has a random magnitude between

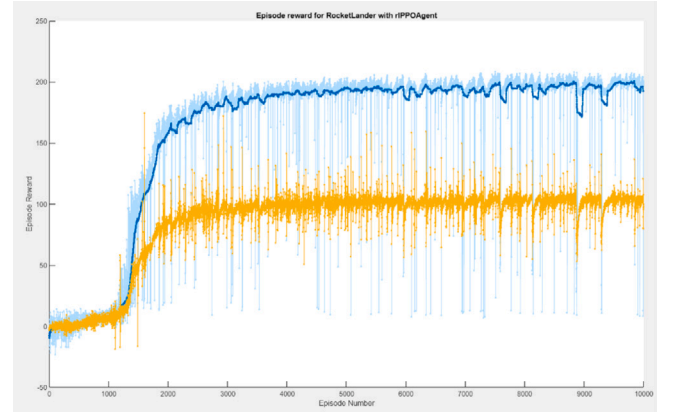


Fig. 20. Learning process and amount of rewards at each stage.

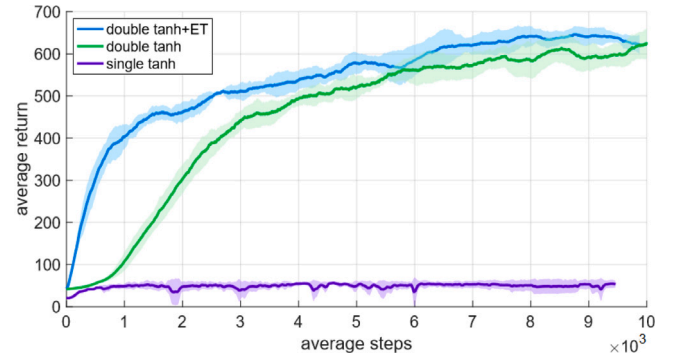


Fig. 21. Comparison of dual tanh activator with single tanh activator.

Table 8

Comparison of learning by adding neural layers.

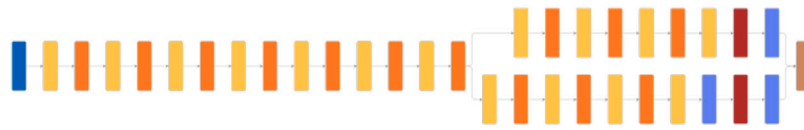
Number of layers	87	79	71	63	51
Training time	120 min	120 min	120 min	120 min	120 min
Number of actor layers	61	55	49	43	35
Number of critic layers	26	24	22	20	16
Episode number	756	854	972	1108	1393
Episode reward	81	76	1682	268	426
Average reward	89	81	174	331	370
Episode Q0	57	44	72	84	87

0 and 1 Newtons on the quadrotor's center of mass. Reinforcement learning training is performed using the PPO method in 10,000 steps. For this part, aerial maneuvers are plotted with their reference paths and the amount of deviation error. This maneuver, called Split S , accelerates the quadrotor at a distance of 10 meters and rolls 180 degrees. Then it enters a half-circle with a radius of $r = 4$ m. Learning on this path is done with 10,000 episodes, the process of which is shown in Fig. 20.

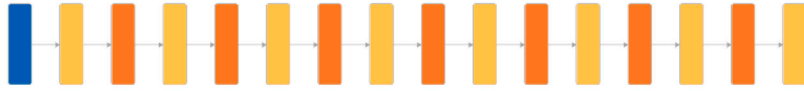
To confirm the importance of the dual activation function $\tanh$ and ET, Fig. 21 is presented. In this figure, it is clear that the quadrotor with the dual activation function $\tanh$ and ET has the best performance compared to other possible performances. Interestingly, the single $\tanh$ activation function has a slight ability to stabilize the quadrotor and does not easily learn any relevant reactions.

Also, adding multiple neural layers with 256 neurons between the previous 196 neuron layers in both the operator and critic networks results in changes in reward and learning time in 2000 episodes as follows in 22:

In the following, all the desired information in these two hours will be examined in the Table 8.



(a) Actor network with 35 layers



(b) Critic network with 16 layers

Fig. 22. Actor and critic networks used, this figure illustrates the impact of integrating additional neural layers with 256 neurons.

The Table 8 provides a comparative analysis of neural network performance with different layer configurations. The training time remains constant at 120 min for all setups, indicating a standard training period. As the total number of layers decreases from 87 to 51, there is a trend towards simpler architectures. Actor layers are reduced from 61 to 35, and critic layers from 26 to 16, suggesting a proportional decrease in complexity within the actor–critic model components. The episode number increases from 756 to 1393, implying more iterations are needed as the network is simplified. The episode reward varies, but the average reward generally increases with fewer layers, reaching a peak of 370 with 51 layers, indicating that a simpler model may yield more consistent rewards. The initial Q-value of the episode, Episode Q0, shows a slight increase from 57 to 87, suggesting improved initial estimations by the Q-learning algorithm as the network becomes less complex. In summary, reducing the number of layers does not negatively impact training time and seems to lead to more consistent rewards and better initial value estimations in this experiment. The data also underscores the actor–critic model's adaptability to varying network complexities.

According to the results obtained in two hours, it is observed that a neural network with more layers is slower to train but more accurate and receives a higher reward. It is also possible that increasing the number of neural network layers may increase the complexity of the model and require more training data.

5. Experimental results

5.1. Test bench

To have a secure and reliable verification, an experimental test stand is manufactured to evaluate the performance of the controller in the quadrotor (Fig. 23).

The test stand allows the quadrotor to have roll and pitch motion up to $\pm 30^\circ$ and to have yaw motion freely. A quadrotor is installed in the stand and an IMU is used to obtain the acceleration and the velocity information of the system. An arduino microcontroller is used as an onboard controller to control rotors speed and a GY-80 sensor is used for measuring the value of roll pitch and yaw. Fig. 24 shows the simple system architecture of the quadrotor. The microcontroller reads analog signals from accelerometers and gyroscopes. The signals should be converted to the digital form to be processed by the microcontroller. To measure the inclination of the quadrotor for pitch and roll, the digital data will be combined. To prevent miscalculation which will affect the



Fig. 23. The quadrotor robot on the test stand.

stability in quadrotor flight a suitable filter is needed for reducing the noise from sensor output to get a near-real value. The filtered data will be used as feedback in the PID control system. Kalman Filter is used to reduce the noise from the sensor. The microcontroller produces four pulses with modulations (PWM) to control the brushless motor speed. These pulses will be sent to the electronic speed controllers (ESC) which will control the brushless motors.

The Kalman Filter significantly contributes to the stability of the quadrotor's flight by processing sensor data. It does so by integrating the outputs from the accelerometer, gyroscope, and compass sensors, which are prone to noise due to system vibrations caused by the motion of the rotors. The Kalman Filter effectively compensates for this noise, ensuring that the data used to control the angular position of the quadrotor is accurate and reliable. By providing a more precise estimation of the system's position, the Kalman Filter allows for better control actions from the PID controller. This results in a more stable flight with less settling time and reduced maximum overshoot in response to step inputs, as compared to a classical PID controller. In essence, the Kalman Filter is crucial for filtering out the noise and improving the quality of sensor data, which is vital for maintaining the stability and control of the quadrotor during flight.

Kalman Filter is used to reduce the noise from the sensor. The microcontroller produces four pulses with modulations (PWM) to control the brushless motor speed. These pulses will be sent to the electronic speed controllers (ESC) which will control the brushless motors. Table 9 lists the details of the quadrotor components.

Digital and analog sensors are two types of IMU which are used in robotic industry. In the paper, the analog type is used to measure the attitude position of the quadrotor. The analog sensor gives voltage

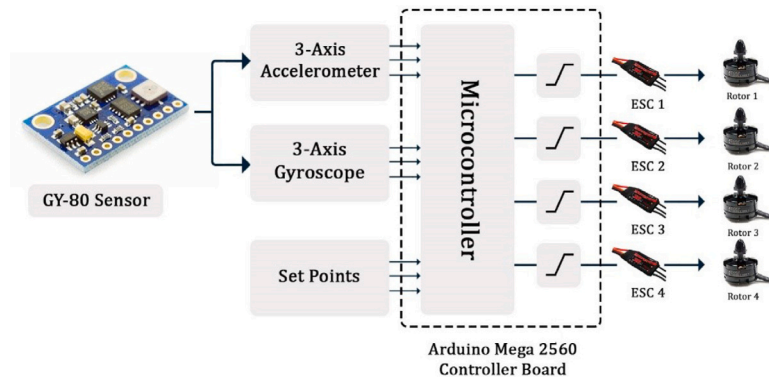


Fig. 24. The experimental system block diagram.

Table 9

Quadrotor component details used in the testbench and real robot.

Component's name	Model	Manufacturer	Description
Motor	MT3510	Emax	Max thrust: 1780 G
Electric speed controllers	Simon series	Emax	Continuous current: 12 A
Accelerometer	ADXL345	–	Chipset of GY-80 sensor
Gyroscope	L3G4200D	–	Chipset of GY-80 Sensor
Propeller	10 × 4.5L	–	–

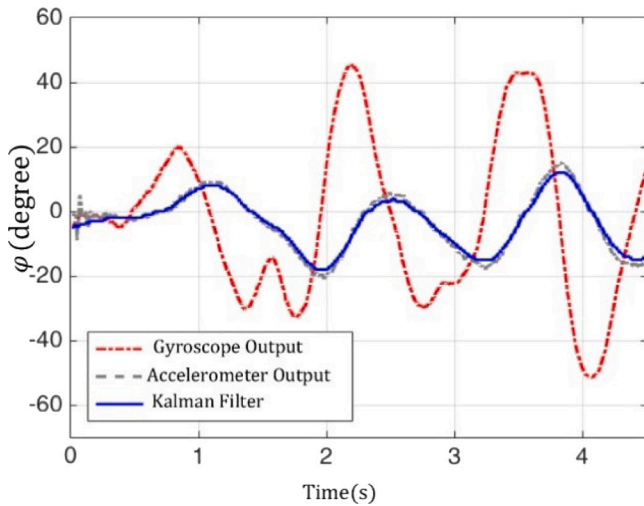


Fig. 25. Sensor and Kalman filter output in the case of off motors.

level to the microcontroller and the voltage will convert the value to the digital value by ADC module. To estimate the attitude of the quadrotor, Kalman filter is used to combine the output values of accelerometer and gyroscope. As shown in Fig. 25 the accelerometer output and the Kalman filter output are near to each other while motors are off and the angle changes by movement of hand.

When the motors are on, noises will appear in the output of sensors, Kalman filter is used to reduce the noises by combination of accelerometer and gyroscope output. As shown in Fig. 26 Kalman filter could remove the noises properly.

5.2. Fuzzy experimental results

In order to investigate the fuzzy-PID controller performance in attitude control of quadrotor, this controller was implemented in the experimental model with three degrees of freedom. Several experiments were conducted to evaluate the performance and efficiency of this type of controller. In the first test, the controller controls the system

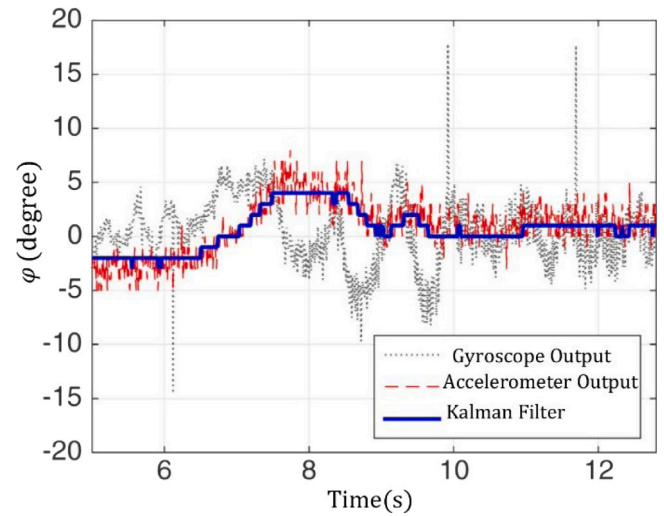


Fig. 26. Efficacy of Kalman filter in noise reduction for sensor outputs.

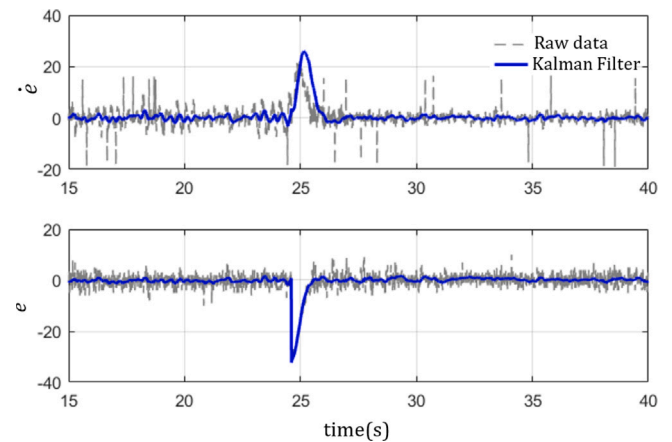


Fig. 27. Performance analysis of fuzzy-PID controller in quadrotor attitude adjustment.

to change the angle of the Roll from 10 to −20 degree. As shown in Fig. 27, for this experiment, the attitude control has been done appropriately. To illustrate Kalman filter function, the output without filter is also shown in the figure.

Error values and error derivatives are considered as inputs of the fuzzy system, and the control gains at any given time are set with respect to these two parameters. In order to evaluate the output of the

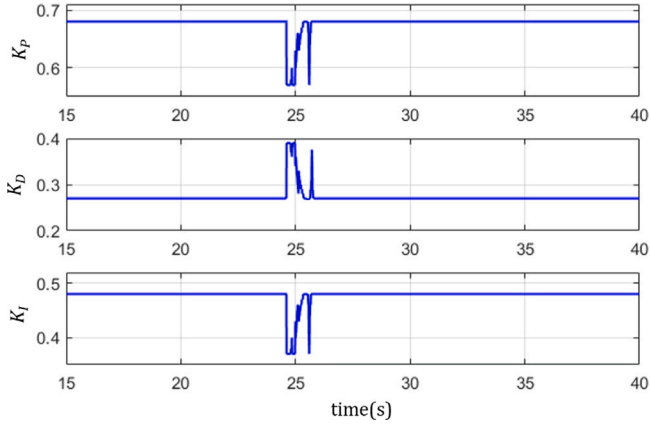


Fig. 28. Dynamic adjustment of control gains in fuzzy system for roll control.

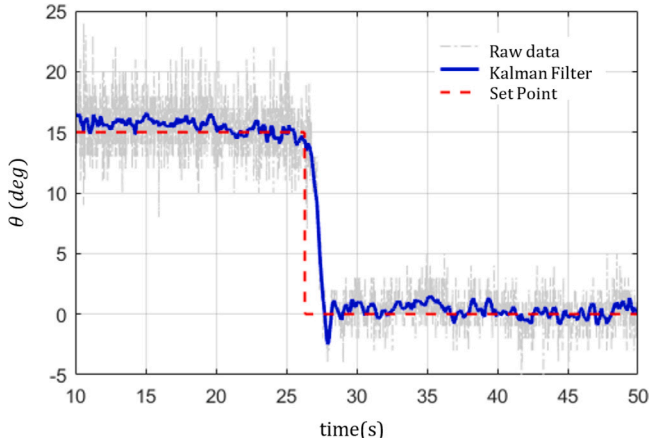


Fig. 29. Rapid and precise pitch angle control via controller output.

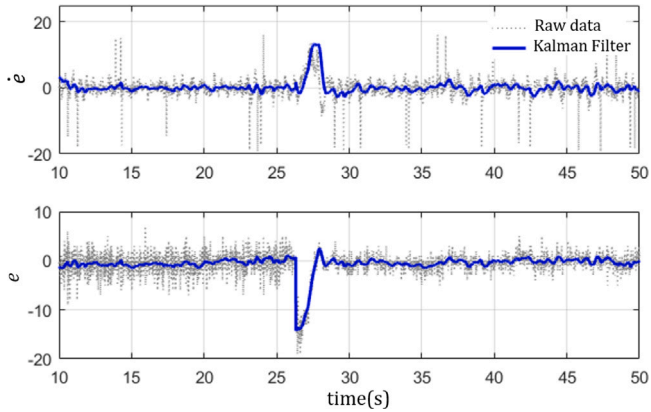


Fig. 30. Variations of error and derived error in the pitch control with fuzzy-PID controller.

fuzzy system during the first experiment, the trend of control gains' changes in Roll control is shown in Fig. 28.

In the second experiment to control the Pitch angle, an angle of 0 degrees is given as the desired angle input to the controller. The result is shown in Fig. 29, where the controller output leads to a fast and accurate appropriate response from the initial angle to the desired angle.

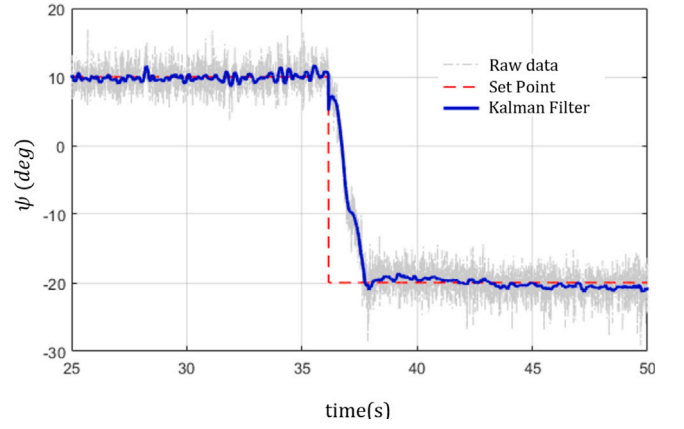


Fig. 31. Experimental results of yaw control using fuzzy PID controller.

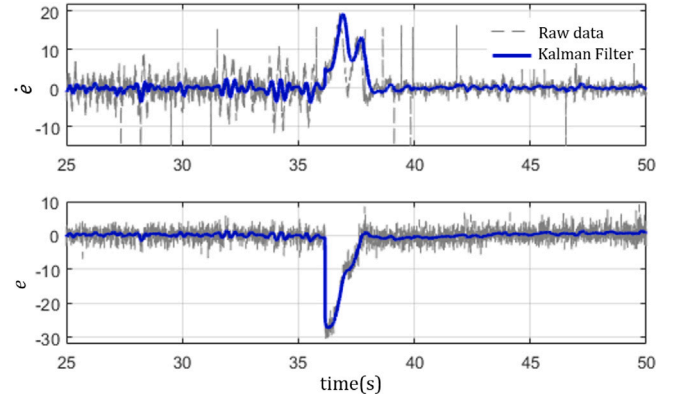


Fig. 32. Variations of error and derived error in the yaw control with fuzzy-PID controller.

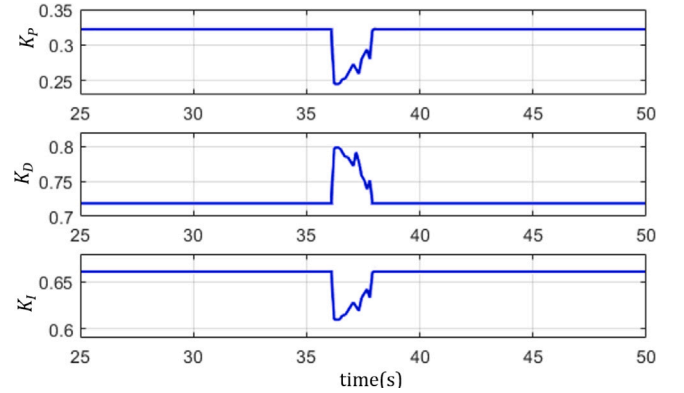


Fig. 33. Variations of gain values in yaw control.

The error variations and the error derivative of attitude control in Pitch direction is shown in Fig. 30.

In the third experiment, the controller performance was evaluated to change the Yaw angle from 10° to the optimal angle of -20°, as shown in Fig. 31.

The trend of error variation and the error derivative in Yaw angle controlling is shown in Fig. 32.

Fig. 33 shows the trend of changes of control gain values over time unit Yaw attitude control.

The sequence of images for roll, pitch and yaw control using fuzzy PID controller is shown in Fig. 34.

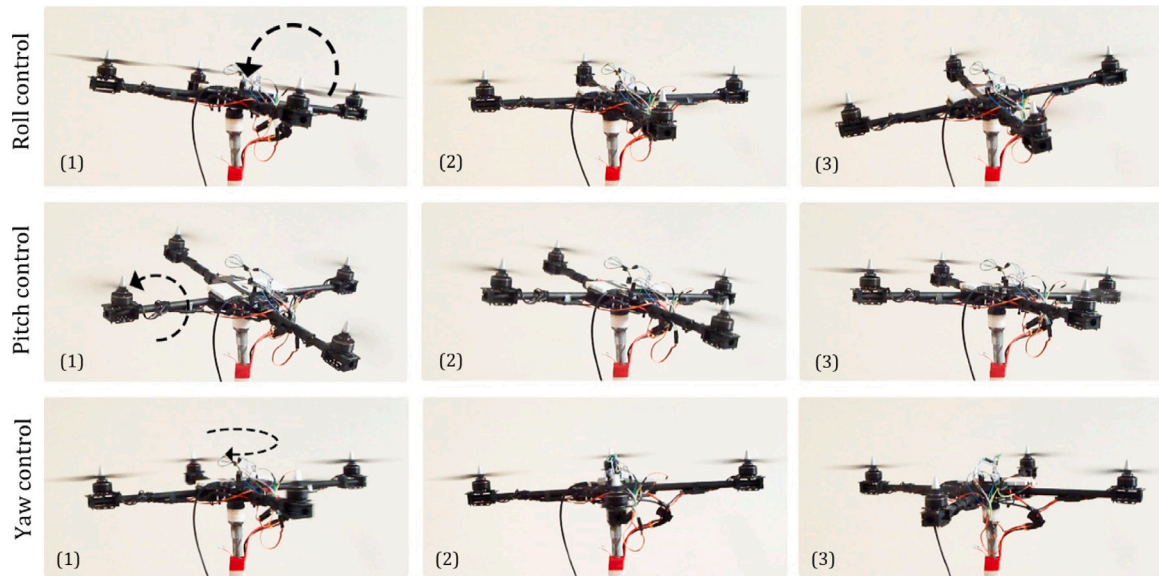


Fig. 34. The sequence of images for roll, pitch and yaw control using fuzzy PID.

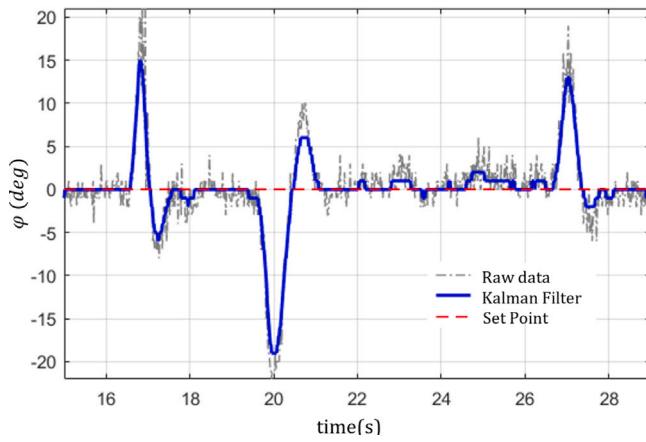


Fig. 35. Assessing controller robustness against random perturbations.

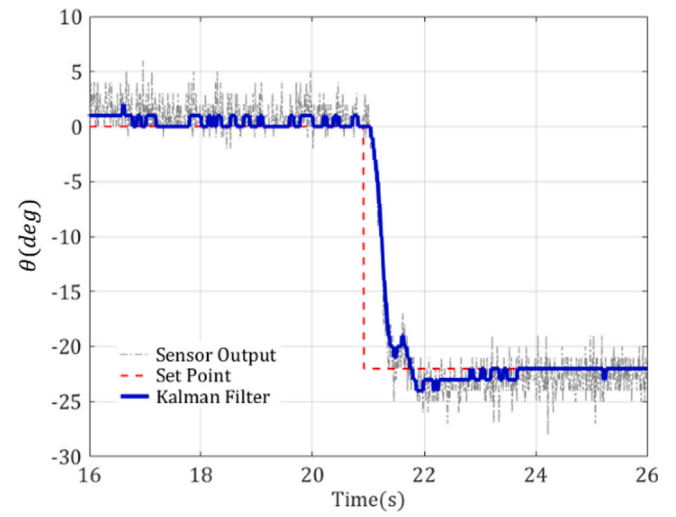


Fig. 37. Pitch angle control experiment results using FOPID controller.

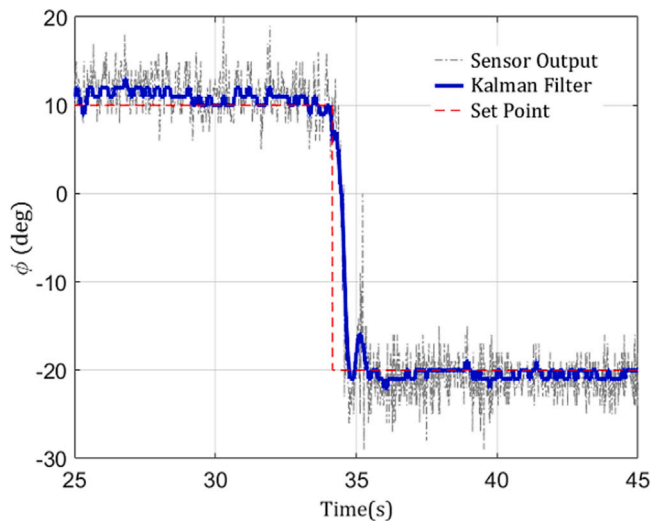


Fig. 36. Demonstrating the FOPID controller's attitude control in action.

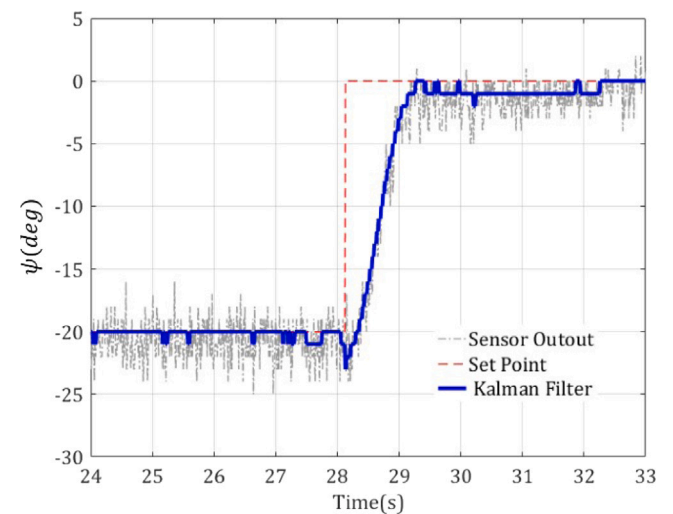


Fig. 38. Yaw angle control experiment results using FOPID controller.

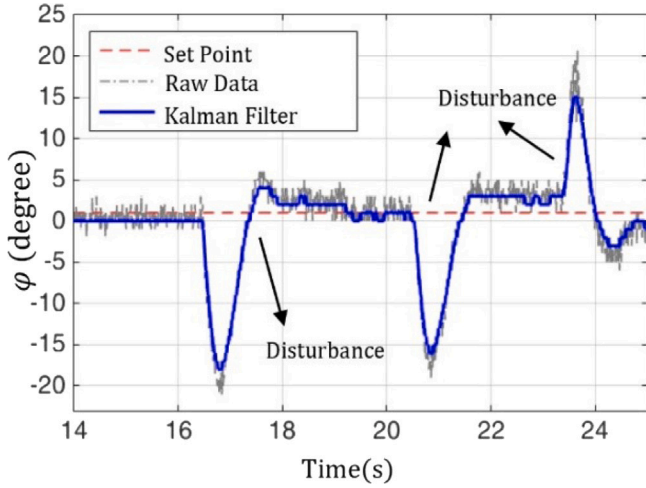


Fig. 39. Roll angle control with FOPID controller while subjecting to disturbance.

In order to evaluate the robustness of the designed controller, several random perturbations have been entered into the system. As shown in Fig. 35, the controller maintains the system's stability at an optimum speed.

5.3. FOPID results

In the experiment, FOPID controller is used to control the attitude of the experimental model. Various experiments were performed to check the performance and the effectiveness of FOPID controller. In experiment 1, FOPID controller controls the roll angle while the quadrotor starts at 10° and ends at the goal roll angle of -20° . As the result of the experiment is presented in Fig. 36, the quadrotor performs the desired motion successfully.

The pitch angle during the experiment is shown in Fig. 37 while quadrotor starts at 0° and ends at the desired angle of -22° .

As shown in Fig. 38 FOPID controller controls the yaw angle from -20° and to the goal of 10° . Finally, to check the robustness of the

FOPID controller to the external disturbances, random disturbances are applied to the quadrotor. As shown in Fig. 39 the controller stabilized the quadrotor rapidly even with disturbance is applied.

The sequence of images in the experiments are shown in Fig. 40

5.4. Flight experiments

The final version of the layered controller is implemented on a quadrotor drone similar to the test-bench. The main experiment is autonomous loop path for the drone. In this task, the flying robot should control the attitude and orientation while slowly increasing its height and x-coordination in a loop. This job is complicated because three different parameters should be simultaneously controlled by the system. Fig. 41 demonstrates the final flying drone. The autopilot build based on layered learning structure is installed on the drone. Some snapshots of the autonomous loop experiment are presented in Fig. 41. In the figure, the robot is first on the ground. In the last part of the picture, the robot is successfully in the air in a loop path.

The challenges associated with controlling three different parameters simultaneously in the autonomous loop path experiment include managing the **attitude, orientation, and incremental changes in height and x-coordination**. These parameters must be precisely controlled to ensure the quadrotor's stability and accurate navigation along the loop path. The complexity lies in the need for real-time adjustments and synchronization of these parameters to maintain a smooth and stable flight trajectory.

The **layered learning structure** addresses these challenges by dividing the control system into layers, each responsible for different aspects of the flight control. This modular approach allows for specialized processing and decision-making at each layer, which can then be integrated to produce a cohesive control strategy. By doing so, the layered learning structure enables the quadrotor to:

- **Simultaneously Process Multiple Control Parameters:** Each layer can focus on a specific control parameter, allowing for simultaneous adjustments without overwhelming a single control system.
- **Adapt to Dynamic Conditions:** The layered approach provides flexibility, enabling the system to quickly adapt to changes in one parameter without disrupting the others.

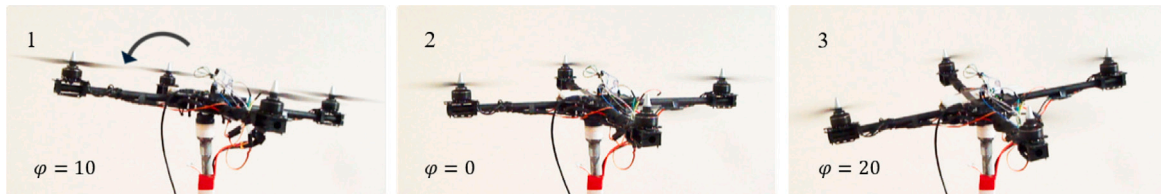


Fig. 18. The sequence of images in the roll angle control

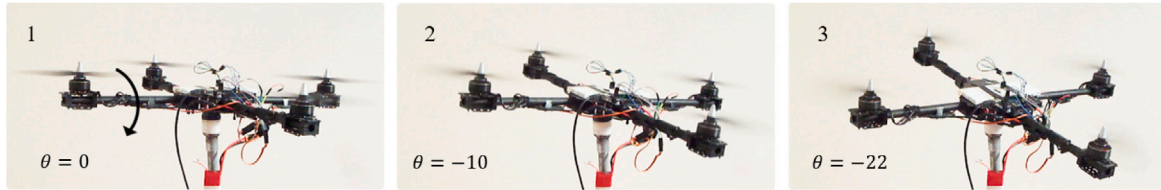


Fig. 19. The sequence of images in the pitch angle control



Fig. 40. The sequence of images control on the test bench, captures the PID controller's deft handling of the yaw angle.

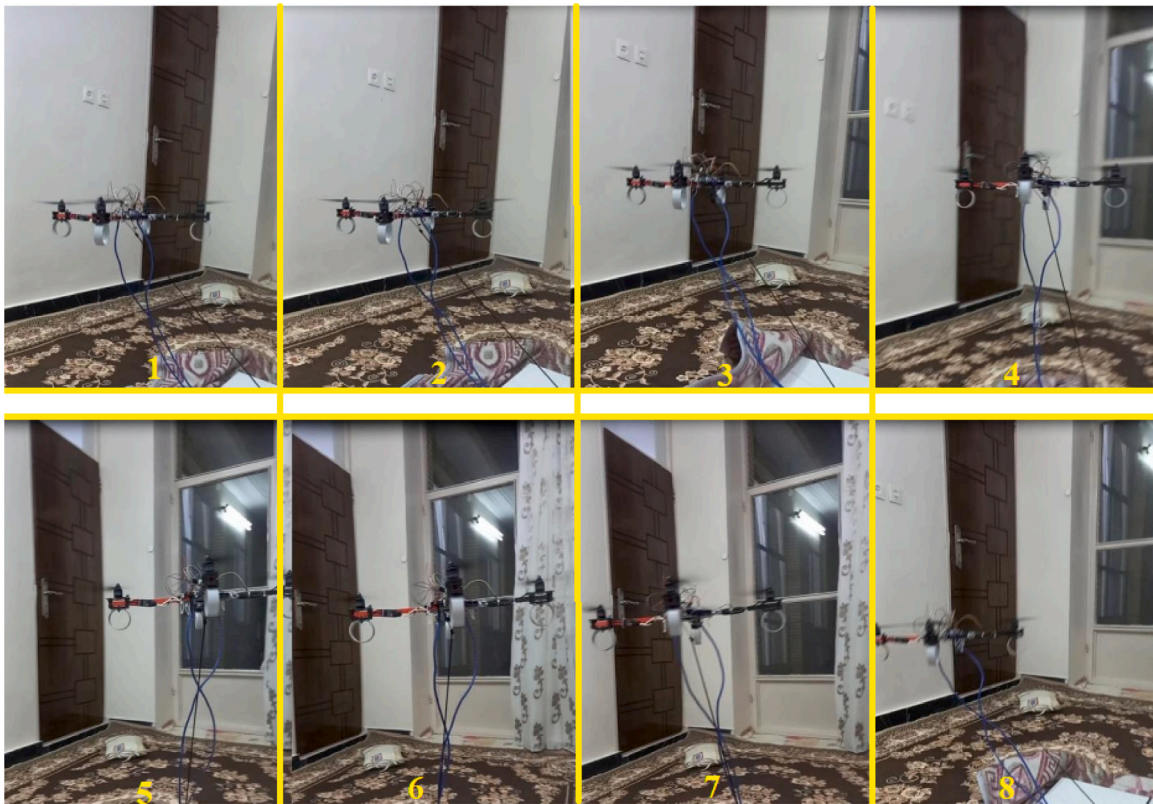


Fig. 41. Ascending to new heights: The autonomous loop path of a layered controller quadrotor.

- **Enhance Decision-Making:** With separate layers working in harmony, the system can make more informed decisions based on a comprehensive understanding of the current state of the quadrotor.
- **Improve Response Time:** Layered learning can reduce the response time by allowing parallel processing of control actions, leading to a more agile and responsive flight.

Overall, the layered learning structure facilitates the management of complex control tasks by distributing them across multiple layers, each with its own specialized function, resulting in a more robust and efficient control system for the quadrotor.

6. Conclusion

Findings: This research introduced a layered learning structure for training a quadrotor, utilizing a PID controller enhanced by a fuzzy system to dynamically adjust control gains. The controllers were optimized using the PSO algorithm to achieve an optimal response. Simulations conducted in MATLAB confirmed that the fuzzy PID controller outperforms the classical PID controller by reducing settling time and overshoot in response to step inputs. Additionally, the integration of sensor outputs through a Kalman filter effectively mitigated system noise, providing accurate position estimations for control purposes.

Research Limitations: The study's limitations include the reliance on simulations for initial evaluations, which may not fully capture the complexities of real-world dynamics. The laboratory model, while effective, may not replicate all the environmental variables that could impact the quadrotor's performance. Furthermore, the optimization algorithms used, PSO and GA, have inherent limitations in terms of convergence speed and the potential for getting trapped in local optima.

Recommendations for Future Research: Future research should focus on real-world testing to validate the simulation results and further refine the control system. Exploring additional optimization algorithms

could provide insights into achieving even more efficient control gain tuning. The development of more advanced noise compensation techniques would also be beneficial. Moreover, expanding the layered learning structure to incorporate other intelligent control strategies could enhance the quadrotor's adaptability and performance in more complex environments.

CRediT authorship contribution statement

Hamed Shahbazi: Writing – review & editing, Supervision, Software, Methodology, Data curation, Conceptualization. **Vahid Tikani:** Writing – original draft, Software, Resources, Methodology, Conceptualization. **Roholamin Fatahi:** Writing – original draft, Validation, Software, Resources.

Declaration of competing interest

All authors have participated in (a) conception and design, or analysis and interpretation of the data; (b) drafting the article or revising it critically for important intellectual content; and (c) approval of the final version.

This manuscript has not been submitted to, nor is under review at, another journal or other publishing venue.

The authors have no affiliation with any organization with a direct or indirect financial interest in the subject matter discussed in the manuscript

Data availability

The data that has been used is confidential.

References

- Abbasi, S., Khalili, H.A., Daneshmand-Mehr, M., Hajiaghaei-Keshteli, M., 2022. Performance measurement of the sustainable supply chain during the COVID-19 pandemic: A real-life case study. *Found. Comput. Decision Sci.* 47 (4), 327–358.
- Altug, E., Secchi, C., Fantuzzi, C., 2011. Quadrotor modeling and control using Newton-Euler formulation. *J. Intell. Robot. Syst.* 61 (1–4), 209–228.
- Asignacion, Jr., A., Satoshi, S., 2024. Historical and current landscapes of autonomous quadrotor control: An early-career researchers' guide. *Drones* 8 (3), 72.
- Bora, J.P., Kumar, R., Singh, S., 2016. Hybrid fuzzy controller for quadrotor UAV. *J. Intell. Robot. Syst.* 81 (1), 3–17.
- Bouabdallah, S., 2007. Design and Control of Quadrotors with Application to Autonomous Flying. Tech. rep., Epfl.
- Çağlayan, N., Abbasi, S., Yilmaz, I., Erdebili, B., et al., 2024. Bibliometric analysis on the distributed decision, decentralized decision, and fuzzy logic. *Discrete Dyn. Nat. Soc.*
- Chandrasekar, A., Radhika, T., Zhu, Q., 2022. Further results on input-to-state stability of stochastic Cohen-Grossberg BAM neural networks with probabilistic time-varying delays. *Neural Process. Lett.* 1–23.
- de Oliveira Evald, P.J.D., Aoki, V.M., da Silva, C.B., Cardoso, D.S., Pinheiro, P.M., da Costa Botelho, S.S., Junior, P.L.J.D., 2024. A review on quadrotor attitude control strategies. *Int. J. Intell. Robot. Appl.* 1–21.
- Dorcak, L., Petras, I., Kostial, I., Terpak, J., 2002. State-space controller design for the fractional-order regulated system. *arXiv preprint math/0204189*.
- Dunfield, K.A., Karaman, S., Tedrake, R., 2015. Neural network-based controller for quadrotor UAVs. *IEEE Trans. Robot.* 31 (4), 866–880.
- Gaig, Z.-L., 2004. A particle swarm optimization approach for optimum design of PID controller in AVR system. *IEEE Trans. Energy Convers.* 19 (2), 384–391.
- Gonzalez, E.D.S., Abbasi, S., Azhdarifard, M., 2023. Designing a reliable aggregate production planning problem during the disaster period. *Sustain. Oper. Comput.* 4, 158–171.
- Gu, Y., Cheng, S., Hovakimyan, N., 2024. Proto-MPC: An encoder-prototype-decoder approach for quadrotor control in challenging winds. *arXiv preprint arXiv:2401.15508*.
- Idrissi, M., Salami, M., Annaz, F., 2022. A review of quadrotor unmanned aerial vehicles: applications, architectural design and control algorithms. *J. Intell. Robot. Syst.* 104 (2), 22.
- Kennedy, J., Eberhart, R., 1995. Particle swarm optimization. In: *Proceedings of ICNN'95-International Conference on Neural Networks*. Vol. 4, IEEE, pp. 1942–1948.
- Lee, T., Leok, M., McClamroch, N.H., 2010. Sliding mode control of a quadrotor helicopter. *IEEE Trans. Control Syst. Technol.* 18 (1), 16–23.
- Li, G., Tuncnez, A., Loianno, G., 2022. Learning model predictive control for quadrotors. In: *2022 International Conference on Robotics and Automation. ICRA, IEEE*, pp. 5872–5878.
- Liu, X., Chen, X., Li, S., Li, J., 2019. Designing a control system for a completely automatic dynamic system. *J. Intell. Robot. Syst.* 94 (1), 1–10.
- Maaruf, M., Mahmoud, M.S., Ma'arif, A., 2022. A survey of control methods for quadrotor uav. *Int. J. Robot. Control Syst.* 2 (4), 652–665.
- Mirtaba, M., Jeddi, M., Nikoofard, A., Shirmohammadi, Z., 2023. Design and implementation of a low-complexity flight controller for a quadrotor UAV. *Int. J. Dyn. Control* 11 (3), 689–700.
- Ou, C., Lin, W., 2006. Comparison between PSO and GA for parameters optimization of PID controller. In: *2006 International Conference on Mechatronics and Automation. IEEE*, pp. 2471–2475.
- Peng, C., Tian, Y., Bai, Y., Gong, X., Zhao, C., Gao, Q., Xu, D., 2013. ADRC trajectory tracking control based on PSO algorithm for a quad-rotor. In: *2013 IEEE 8th Conference on Industrial Electronics and Applications. ICIEA, IEEE*, pp. 800–805.
- Petrás, I., 2000. The fractional-order controllers: Methods for their synthesis and application. *arXiv preprint math/0004064*.
- Prescott, T.J., Redgrave, P., Gurney, K., 1999. Layered control architectures in robots and vertebrates. *Adapt. Behav.* 7 (1), 99–127.
- Radhika, T., Chandrasekar, A., Vijayakumar, V., Zhu, Q., 2023. Analysis of Markovian jump stochastic Cohen-Grossberg BAM neural networks with time delays for exponential input-to-state stability. *Neural Process. Lett.* 55 (8), 11055–11072.
- Sabatino, F., 2015. Quadrotor control: modeling, nonlinear control design, and simulation.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Sivanandam, S., Deepa, S., Sivanandam, S., Deepa, S., 2008. Genetic algorithm optimization problems. *Introduction to Genetic Algorithms*. Springer, pp. 165–209.
- Stone, P., Veloso, M., 2000. Layered learning. In: *European Conference on Machine Learning*. Springer, pp. 369–381.
- Tarasov, V.E., 2019. On history of mathematical economics: Application of fractional calculus. *Mathematics* 7 (6), 509.
- Vinagre, B.M., Podlubny, I., Dorcak, L., Feliu, V., 2000. On fractional PID controllers: a frequency domain approach. *IFAC Proc. Vol.* 33 (4), 51–56.
- Wang, D., Tan, D., Liu, L., 2018. Particle swarm optimization algorithm: an overview. *Soft Comput.* 22, 387–408.
- Zhang, Z., Yang, H., Fei, Y., Sun, C., Yu, Y., 2023. Control of UAV quadrotor using reinforcement learning and robust controller. *IET Control Theory Appl.* 17 (12), 1599–1610.